String Algorithms

for Competitive Programming

Expert Edition — suffix trees, SA-IS, BWT/FM-index, bit-parallel matching, k-error matching, generalised SAM, palindromic factorisation, and runs

Z.ai

October 22, 2026

Abstract

This is the expert companion to the introductory primer. Where the primer covered the workhorses of competitive programming — KMP, Z, hashing, trie, Aho–Corasick, suffix array, suffix automaton, Manacher, eertree, XOR/persistent tries, Duval/Booth — this volume goes two layers deeper. It adds the online suffix tree of Ukkonen (the missing sibling of the suffix automaton), the linear-time SA-IS suffix-array construction, the Burrows–Wheeler transform and its reversible FM-index (the data structure behind modern compressors and aligners), bit-parallel pattern matching (Shift-And, Shift-Or, BNDM, and Myers' $O(n/w)$ edit-distance algorithm), approximate string matching with k errors (Landau–Vishkin and Myers bit-vector), the generalised suffix automaton for multiple strings, palindromic factorisation via the eertree's series links, and the runs theorem with its algorithmic consequences. Every chapter carries full proofs or proof sketches, a worked example, a C++17 reference implementation, a complexity analysis, and a `When to use` pattern-recognition block. The document closes with a master decision tree and a hybrid-pattern catalogue.

Contents

1 Foundations and the Pattern-Recognition Framework, Revisited **9**
- 1.1 Notation, extended . . . 9
- 1.2 Complexity vocabulary, extended . . . 9
- 1.3 The hybrid-pattern catalogue . . . 9
- 1.4 A note on numerical robustness . . . 10

2 Polynomial Hashing, Randomised Bases, and Anti-Hash Theory **10**
- 2.1 The collision bound . . . 10
- 2.2 Randomised base: the production-ready defence . . . 11
- 2.3 Bidirectional hashing . . . 11
- 2.4 Anti-hash: Thue–Morse in detail . . . 12
- 2.5 Hashing + binary search: longest common substring of k strings . . . 12
- 2.6 When to use hashing, expert edition . . . 12

3 Prefix Function: Automaton View, Periodicity, and Applications **13**
- 3.1 The failure automaton as a DFA . . . 13
- 3.2 Periodicity lemma, with proof . . . 13
- 3.3 Streaming KMP: the text is a stream . . . 14
- 3.4 Worked example: counting distinct factors of X^k . . . 14
- 3.5 When to use the failure automaton . . . 14
- 3.6 Codeforces references . . . 15

4 Z-Function: Two-Sided Pattern Matching and Wildcards **15**
4.1 Two-sided pattern matching . . . 15
4.2 Matching with wildcards via Z (single wildcard) . . . 16
4.3 When to use Z, expert edition . . . 16
4.4 Codeforces references . . . 16

5 Trie and Aho–Corasick: Matrix Exponentiation and Counting **16**
5.1 Reformulation: AC automaton as a DFA . . . 16
5.2 Implementation . . . 17
5.3 Worked example . . . 17
5.4 When to use AC + matrix exponentiation . . . 18
5.5 Codeforces references . . . 18

6 Suffix Array: SA-IS, DC3, and Sparse-Table RMQ **18**
6.1 SA-IS in concept . . . 18
6.2 Linear LCP: Kasai's algorithm, with the amortisation proof . . . 19
6.3 Sparse table for RMQ on LCP, with build complexity . . . 19
6.4 Distinct substrings, with proof . . . 19
6.5 Smallest rotation via SA in $\mathcal{O}(n \log n)$ . . . 19
6.6 When to use SA, expert edition . . . 19
6.7 Codeforces references . . . 20

7 Suffix Automaton: Endpos Tree, Multi-String, and Substring DP **20**
7.1 The suffix-link tree . . . 20
7.2 DP on the suffix-link tree . . . 20
7.3 Number of distinct substrings ending at each prefix . . . 21
7.4 Generalised suffix automaton . . . 21
7.5 Longest common substring of two strings, in $\mathcal{O}(|S_1| + |S_2|)$ . . . 21
7.6 When to use SAM, expert edition . . . 22
7.7 Codeforces references . . . 22

8 Ukkonen's Online Suffix Tree **22**
8.1 Why learn the suffix tree? . . . 22
8.2 Implicit edges and the active point . . . 22
8.3 The three suffix extension rules . . . 23
8.4 Amortised linear time . . . 23
8.5 Implementation sketch . . . 23
8.6 Applications . . . 24
8.7 When to use the suffix tree . . . 24
8.8 Codeforces references . . . 25

9 Generalised Suffix Automaton for Multiple Strings **25**
9.1 Construction . . . 25
9.2 Applications . . . 26
9.3 When to use the generalised SAM . . . 26

10 Burrows–Wheeler Transform and FM-index **26**
10.1 Definition of BWT . . . 26
10.2 Reversibility: the LF mapping . . . 26
10.3 Construction . . . 27
10.4 FM-index: count and locate queries . . . 27
10.5 Wavelet tree: the compressed Occ table . . . 28
10.6 When to use BWT / FM-index . . . 28

10.7 Codeforces references . . . 28

11 Bit-Parallel String Algorithms **28**
11.1 Shift-And / Shift-Or exact matching . . . 28
11.2 BNDM: backward nondeterministic matching . . . 29
11.3 Myers' bit-vector edit distance . . . 29
11.4 When to use bit-parallel . . . 30
11.5 Codeforces references . . . 30

12 Approximate Pattern Matching with k Errors **30**
12.1 Hamming distance k-mismatch . . . 30
12.2 Edit-distance k-differences (Landau–Vishkin) . . . 31
12.3 Myers' bit-vector k-difference . . . 31
12.4 When to use approximate matching . . . 32
12.5 Codeforces references . . . 32

13 Palindromic Factorisation via Eertree Series Links **32**
13.1 Series links: the key data structure . . . 32
13.2 The $\mathcal{O}(n \log n)$ algorithm . . . 32
13.3 When to use the palindromic factorisation . . . 34
13.4 Codeforces references . . . 34

14 Trie Tricks: XOR, Persistent, Merge, and Heavy-Light on Trie **34**
14.1 Heavy-light on trie . . . 34
14.2 Trie + bitmask (alphabet = small set) . . . 34
14.3 Persistent trie applications . . . 34
14.4 When to use expert trie tricks . . . 35

15 Lyndon Words, Duval, Booth, and the Runs Theorem **35**
15.1 Lyndon factorisation, with the full proof sketch . . . 35
15.2 Booth's algorithm: full statement . . . 35
15.3 Runs theorem . . . 35
15.4 Algorithms based on runs . . . 36
15.5 When to use Lyndon / Booth / runs . . . 36
15.6 Codeforces references . . . 36

16 Suffix Array Applications: A Worked-Problem Compendium **36**
16.1 Problem 1: Longest repeated substring . . . 36
16.2 Problem 2: k-th lexicographically smallest substring . . . 36
16.3 Problem 3: Number of distinct substrings . . . 37
16.4 Problem 4: Longest common substring of two strings . . . 37
16.5 Problem 5: Substring frequency queries . . . 37
16.6 Codeforces references . . . 38

17 Suffix Automaton Applications: A Worked-Problem Compendium **38**
17.1 Problem 1: Online substring existence . . . 38
17.2 Problem 2: Number of distinct substrings ending at each prefix . . . 38
17.3 Problem 3: Number of occurrences of each substring . . . 38
17.4 Problem 4: Smallest cyclic rotation in $\mathcal{O}(n)$ without Booth . . . 38
17.5 Problem 5: Distinct substrings of S that occur in T . . . 38
17.6 Codeforces references . . . 39

18 Combinatorics on Words: Necklaces, Primitive Words, and de Bruijn Sequences 39
18.1 Necklaces and Lyndon words . . . 39
18.2 Primitive words . . . 39
18.3 de Bruijn sequences . . . 39
18.4 When to use combinatorics on words . . . 40
18.5 Codeforces references . . . 40

19 Streaming String Algorithms 40
19.1 Pattern matching in a stream . . . 40
19.2 Distinct substrings in a stream (estimation) . . . 41
19.3 Approximate membership in a stream . . . 41
19.4 When to use streaming string algorithms . . . 41

20 Suffix Tree + LCA: $\mathcal{O}(1)$ LCP Queries 41
20.1 Euler tour . . . 41
20.2 Reduction to RMQ . . . 41
20.3 LCP from LCA . . . 41
20.4 Complexity . . . 42
20.5 When to use the suffix tree + LCA approach . . . 42

21 Wavelet Trees and Inverted Indexes 42
21.1 Definition . . . 42
21.2 Operations . . . 42
21.3 Bit-vector rank/select in $\mathcal{O}(1)$ . . . 42
21.4 CP application: Occ table for FM-index . . . 43
21.5 When to use wavelet trees . . . 43

22 String Dynamic Programming: A Compendium 43
22.1 Levenshtein edit distance, with backtracking . . . 43
22.2 Damerau–Levenshtein (with transpositions) . . . 43
22.3 Weighted edit distance . . . 44
22.4 Longest common subsequence (LCS) . . . 44
22.5 Shortest common supersequence . . . 44
22.6 Number of distinct LCS . . . 44
22.7 Regular expression matching . . . 44
22.8 String partitioning DP . . . 44
22.9 DP on cyclic strings . . . 44
22.10 When to use string DP . . . 44
22.11 Codeforces references . . . 45

23 Hashing: Advanced Applications 45
23.1 Application 1: Longest palindromic substring via hashing + binary search . . . 45
23.2 Application 2: Number of distinct substrings of length exactly k . . . 45
23.3 Application 3: Longest common substring of k strings . . . 45
23.4 Application 4: Number of positions where S and T differ, with at most one wildcard 45
23.5 Application 5: Substring equality queries on a dynamic string (with rollback) . . . 46
23.6 When to use hashing, advanced . . . 46

24 Aho–Corasick DP Variants 46
24.1 Variant 1: Count strings of length n containing each P_i at least once . . . 46
24.2 Variant 2: Maximum-score walk through T where each P_i contributes a weight . . . 46
24.3 Variant 3: Shortest string containing all P_i . . . 46
24.4 Variant 4: Number of occurrences of P_i in T . . . 47

24.5 When to use AC DP variants . . . 47
24.6 Codeforces references . . . 47

25 Practice Problems with Full Solutions **47**
25.1 Problem 1: Two-string longest common substring . . . 47
25.2 Problem 2: Number of distinct substrings of S . . . 47
25.3 Problem 3: Minimum palindromic factorisation . . . 48
25.4 Problem 4: Count strings of length n avoiding banned words . . . 48
25.5 Problem 5: Smallest cyclic rotation . . . 48
25.6 Lessons . . . 48

26 Suffix Tree Applications: Worked Problems **48**
26.1 Problem 1: Number of leaves below each internal node . . . 48
26.2 Problem 2: Longest substring occurring at least k times . . . 49
26.3 Problem 3: All distinct substrings starting with P . . . 49
26.4 Problem 4: k-th lexicographically smallest suffix of S . . . 49
26.5 Problem 5: Longest common substring of three strings . . . 49
26.6 When to use the suffix tree for these problems . . . 49

27 Bioinformatics Applications **50**
27.1 Read alignment with k-mismatch . . . 50
27.2 Variant calling: find positions where two genomes differ . . . 50
27.3 de novo assembly: shortest common superstring . . . 50
27.4 de Bruijn graph assembly . . . 50
27.5 When to use bioinformatics-style string algorithms . . . 50
27.6 Codeforces references . . . 51

28 Bit-Parallel Algorithms: Worked Examples **51**
28.1 Example 1: Bit-parallel LCS . . . 51
28.2 Example 2: Bit-parallel regex matching (Shift-Or on NFA) . . . 51
28.3 Example 3: Bit-parallel palindrome substring detection . . . 51
28.4 Example 4: Bit-parallel subset enumeration on small alphabet . . . 51
28.5 When to use bit-parallel examples . . . 51

29 Approximate Matching: Worked Examples **52**
29.1 Problem 1: Pattern with character classes and k-mismatch . . . 52
29.2 Problem 2: Approximate pattern matching with gaps . . . 52
29.3 Problem 3: Edit distance with bounded workspace . . . 52
29.4 When to use these variants . . . 52

30 String Hashing with FFT and Convolution **52**
30.1 Pattern matching with wildcards via FFT . . . 52
30.2 Edit distance with wildcards via FFT . . . 53
30.3 When to use FFT-based string algorithms . . . 53
30.4 Codeforces references . . . 53

31 Multiple Pattern Matching: A Comparison **53**
31.1 Aho–Corasick . . . 53
31.2 Suffix array + binary search per pattern . . . 53
31.3 Hashing + verification . . . 54
31.4 Comparison and decision rule . . . 54

32 Advanced Periodicity: Fine–Wilf and Beyond **54**
32.1 Fine–Wilf theorem . . . 54
32.2 Periodicity lemma corollaries . . . 54
32.3 Border array and periodicity . . . 55
32.4 Critical factorisation theorem . . . 55
32.5 When to use advanced periodicity . . . 55

33 String Compression Algorithms **55**
33.1 Run-length encoding (RLE) . . . 55
33.2 Lempel–Ziv–Welch (LZW) . . . 55
33.3 Burrows–Wheeler transform + Move-to-Front (BWT + MTF) . . . 55
33.4 RePair grammar compression . . . 56
33.5 LZ77 sliding-window . . . 56
33.6 When to use string compression . . . 56
33.7 Codeforces references . . . 56

34 Squares, Repetitions, and Maximal Runs **56**
34.1 Number of squares . . . 56
34.2 Number of runs . . . 56
34.3 Algorithm: enumerate all runs in $\mathcal{O}(n)$ . . . 57
34.4 All squares via runs . . . 57
34.5 When to use squares / runs . . . 57

35 Suffix Automaton: Advanced Topics **57**
35.1 SAM as the minimal DFA accepting suffixes . . . 57
35.2 Endpos tree topology . . . 57
35.3 Substring DP on SAM . . . 57
35.4 Computing the smallest lexicographic substring with property P . . . 58
35.5 Online SAM operations . . . 58
35.6 When to use SAM advanced topics . . . 58

36 Order Statistics on Strings **58**
36.1 k-th lexicographically smallest suffix . . . 58
36.2 k-th lexicographically smallest substring . . . 58
36.3 k-th lexicographically smallest cyclic rotation . . . 58
36.4 k-th lexicographically smallest string of length n avoiding patterns . . . 58
36.5 When to use order statistics . . . 59

37 Trie Variants: PATRICIA, Compressed Trie, Radix Trie **59**
37.1 PATRICIA trie . . . 59
37.2 Compressed trie . . . 59
37.3 Radix trie . . . 59
37.4 When to use trie variants . . . 59

38 Distributed and Parallel String Algorithms **59**
38.1 Parallel suffix array construction . . . 59
38.2 Distributed BWT via MapReduce . . . 60
38.3 Parallel Aho–Corasick . . . 60
38.4 When to use distributed / parallel . . . 60

39 Open Problems and Research Directions **60**
39.1 Tight bound on the number of distinct squares . . . 60
39.2 Optimal suffix-array construction in the cache-aware model . . . 60
39.3 Compressed text indexing with constant edit operations . . . 60
39.4 Edit distance in truly subquadratic time . . . 60
39.5 Hard problems in pattern avoidance . . . 61
39.6 When to engage with open problems . . . 61

40 Complexity Reference Card **61**
40.1 Time complexities . . . 61
40.2 Space complexities in perspective . . . 62
40.3 Heuristic for choosing algorithm . . . 62

41 Common Pitfalls and Debugging String Algorithms **62**
41.1 Pitfall 1: Off-by-one in substring hash extraction . . . 62
41.2 Pitfall 2: Overflow in polynomial hash multiplication . . . 62
41.3 Pitfall 3: Forgotten separator in $P\#T$ . . . 63
41.4 Pitfall 4: Suffix array construction with `std::sort` is too slow . . . 63
41.5 Pitfall 5: SAM map-based transitions are too slow for $|\Sigma| = 26$ . . . 63
41.6 Pitfall 6: Eertree's two roots are confused . . . 63
41.7 Pitfall 7: BWT construction uses the wrong sentinel . . . 63
41.8 Pitfall 8: Myers bit-vector edit distance with negative numbers . . . 63
41.9 Pitfall 9: Aho–Corasick output links missed . . . 63
41.10 Pitfall 10: Booth's algorithm with `s + s` doubling . . . 63
41.11 Pitfall 11: SA-IS recursion does not reduce alphabet size . . . 64
41.12 Pitfall 12: Series-link eertree DP visits states in wrong order . . . 64
41.13 Debugging checklist . . . 64

42 Affine Gap and Advanced Edit Distance **64**
42.1 Affine gap definition . . . 64
42.2 Gotoh's algorithm . . . 64
42.3 Bit-parallel affine gap . . . 65
42.4 When to use affine gap . . . 65

43 Online Suffix Tree Updates **65**
43.1 Appending a character . . . 65
43.2 Prepending a character . . . 65
43.3 Replacing a character . . . 65
43.4 Deleting a character . . . 65
43.5 When to use online updates . . . 65

44 Worked Examples in Detail: Three Codeforces Problems **66**
44.1 Example 1: CF 1202E — You Are Given Some Strings . . . 66
44.2 Example 2: CF 932G — Palindrome Partition . . . 67
44.3 Example 3: CF 235C — Cyclical Quest . . . 67
44.4 Lessons from the three examples . . . 67

45 Pattern Recognition Drill: Twenty Quick-Fire Problems **67**
45.1 How to use the drill . . . 68

46 Comparative Survey: SAM vs SA vs Suffix Tree **68**
46.1 Size and construction cost . . . 68
46.2 Memory usage . . . 69
46.3 What each structure is best at . . . 69
46.4 What each is worst at . . . 69
46.5 Decision rule . . . 69

47 Implementation Tips and CP Contest Strategy **69**
47.1 What to have in your template file . . . 70
47.2 Order of attempts . . . 70
47.3 Stress testing . . . 70
47.4 Time-limit management . . . 70
47.5 Common time-limit pitfalls . . . 71

48 Suffix-Link Tree Heavy-Light Decomposition **71**
48.1 Why HLD on the suffix-link tree? . . . 71
48.2 Heavy-light decomposition . . . 71
48.3 Application: per-prefix distinct substring count with constraints . . . 71
48.4 When to use HLD on SAM . . . 71

49 Epilogue: From CP to Research **71**
49.1 What you should now be able to do . . . 72
49.2 Where to go from here . . . 72
49.3 Final thought . . . 72

50 Glossary of String Algorithm Terms **72**
50.1 A–E . . . 72
50.2 F–N . . . 73
50.3 P–Z . . . 74

51 Notation Summary **74**
51.1 Strings and characters . . . 75
51.2 Structures . . . 75
51.3 Complexity notation . . . 75

52 Master Cheat Sheet and Decision Tree **75**
52.1 Decision table . . . 76
52.2 Decision flowchart (text form) . . . 76
52.3 Hybrid problems cheat sheet . . . 77
52.4 Reading order for the expert . . . 77

Annotated Bibliography **78**

1 Foundations and the Pattern-Recognition Framework, Revisited

This chapter is short on algorithms and long on philosophy. The expert edition assumes you have read the introductory primer or already know KMP, hashing, suffix arrays, and suffix automata. We restate the notation, extend the complexity vocabulary to amortised and persistent costs, and crucially add a meta-level tool: the *hybrid-pattern catalogue* — a way of reading a problem statement that recognises when no single algorithm suffices and two must be composed.

1.1 Notation, extended

We retain the primer's conventions: S is a string, $n = |S|$, $S[i]$ is the i-th character with 0-indexing, $S[l\,..\,r]$ is the inclusive substring, and $S[l\,..]$ is the suffix starting at l. We add three notations used throughout the expert edition:

- $\mathrm{rot}_k(S) = S[k\,..]\,S[\,..k-1]$ is the k-th cyclic rotation of S (concatenating the suffix from k with the prefix ending at $k-1$).
- $\mathrm{rev}(S) = S[n-1]S[n-2]\ldots S[0]$ is the reversal.
- S^{∞} denotes the infinite periodic extension $S\,S\,S\ldots$; this is used only for analysis, never stored.
- $\mathrm{lcp}(X, Y)$ is the length of the longest common prefix of X and Y.
- $\mathrm{suffix}(S, i)$ is the suffix $S[i\,..]$; we sometimes write it as S_i in chapters where the running time depends on the suffix count.
- Σ is the alphabet, $\sigma = |\Sigma|$.
- For a state machine (automaton), $|Q|$ is the number of states and $|\delta|$ is the number of transitions.

1.2 Complexity vocabulary, extended

Expert-level string algorithms live and die by constants. We distinguish:

- **Worst-case time**: the standard $T(n)$. For example, suffix array via sort-by-prefix is $T(n) = \Theta(n \log^2 n)$.
- **Amortised time**: many string algorithms (KMP, Z, eertree, suffix automaton) are amortised linear — the cost per character is $\mathcal{O}(1)$ on average, but a single character can take $\Theta(n)$ in the worst case. The total across n characters is $\Theta(n)$.
- **Expected time**: hashing is $\mathcal{O}(n)$ expected, with $1/p$ probability of failure per query. Expert authors break this by choosing S adversarially after seeing the base b.
- **Persistent time**: a persistent trie insert is $\mathcal{O}(\log V)$, but reading a single value at version i is also $\mathcal{O}(\log V)$; the cost is *not* amortised across versions.
- **I/O-aware time**: for $n \gg 10^7$ the dominant cost is cache misses. Array-based transitions beat `map`-based transitions not by a log but by a constant 10–50×.

1.3 The hybrid-pattern catalogue

A hybrid pattern is a problem signature that calls for two (or more) structures. We name them now and refer back to this catalogue throughout:

Name	Composition	Problem signature
AC + DP	Aho–Corasick automaton + DP over (position, state)	"Count strings of length n over Σ avoiding any banned pattern P_i."
SA + sparse table	Suffix array + LCP + RMQ	"For each pair of indices $(l_1, r_1), (l_2, r_2)$, is $S[l_1..r_1] < S[l_2..r_2]$?"
SAM + suffix-link tree DP	Suffix automaton + tree DP	"Number of distinct substrings of S that occur in T" (generalised SAM).
Hash + binary search	Polynomial hashing + binary search on length	"Longest common substring of $k \geq 2$ strings."
Eertree + series-link DP	Eertree + DAG DP over series links	"Minimum palindromic factorisation of S."
BWT + FM-index	Burrows–Wheeler transform + wavelet tree	"Count occurrences of P in S in $\lvert P\rvert \log \sigma$ with $\mathcal{O}(n)$ space at 1 byte per symbol."
Bit-parallel + edit DP	Bit-vector of states + Myers trick	"Edit distance in $\mathcal{O}(n/w)$ where w is word size."
Trie + small-to-large merge	Trie on tree paths + merge technique	"Count distinct substrings over all root-to-leaf paths of a tree T with edge labels."

The point of naming these is that experienced CP solvers recognise hybrids instantly. When you see "count strings of length n avoiding banned words", you should not waste ten minutes inventing AC + DP from scratch — you should write down the AC template, dimension the DP table as $[n][|Q|]$, and fill in the recurrence. The remainder of this book is, in effect, an attempt to teach you to recognise these nine hybrids on sight.

1.4 A note on numerical robustness

Many expert algorithms rely on 64-bit modular arithmetic. Use `unsigned long long` (which is `uint64_t` on modern systems) and let unsigned overflow do the modular reduction modulo 2^{64} for low-stakes hashing. For prime-modulus hashing, use `__int128` to perform the multiplication without overflow (as shown in the primer) and the Mersenne trick for $p = 2^{61} - 1$. We will see one further numerical subtlety in Chapter 11: the bit-parallel edit distance requires *signed* arithmetic to compute $+1/-1/0$ in parallel across a word, and getting the carry-chain right is the difference between a passing and a failing submission.

2 Polynomial Hashing, Randomised Bases, and Anti-Hash Theory

The primer gave you a double-hash structure that is enough for 99% of CP problems. This chapter goes two layers down: it explains why single-modulus hashing breaks, how a *randomised* base defeats precomputed hacks, and how to derive the collision bound rigorously.

2.1 The collision bound

Theorem 2.1 (Collision probability)**.** *For a polynomial hash H modulo prime p with base b chosen uniformly at random from $\{1, \ldots, p-1\}$, the probability that two distinct strings $X \neq Y$ of length $\leq L$ collide is at most L/p.*

Proof sketch

The condition $H(X) \equiv H(Y) \pmod p$ is equivalent to $\sum_i (X[i] - Y[i]) b^{L-1-i} \equiv 0 \pmod p$. The polynomial on the left has degree at most $L-1$ in b, so it has at most $L-1$ roots modulo the prime p. A random b is therefore a root with probability $\leq (L-1)/p < L/p$.

Summing over the q queries gives Pr[any collision] $\le qL/p$.

With $p = 2^{61} - 1 \approx 2 \cdot 10^{18}$ and $L, q \le 10^6$, this gives Pr[collision] $< 10^{-6}$, which is borderline. The defence is *double hashing*: two independent random bases b_1, b_2 (same prime is fine) reduce the bound to $(qL/p)^2 \approx 10^{-12}$, well below the noise floor of round-off error in the rest of your code.

2.2 Randomised base: the production-ready defence

A fixed base $b = 131$ is breakable by the Thue–Morse hack. A *randomised* base chosen at runtime from a seed the author cannot predict is essentially unbreakable: the author cannot precompute the test before seeing your seed.

Randomised double hash with runtime seed

```cpp
#include <chrono>
struct SafeHash {
    static const uint64_t M1 = 1'000'000'007ULL;
    static const uint64_t M2 = 1'000'000'009ULL;
    uint64_t b1, b2;          // randomised base, fixed per run
    std::vector<uint64_t> h1, h2, pw1, pw2;

    SafeHash(const std::string& s) {
        // seed with high-resolution clock; author cannot predict
        std::mt19937_64 rng(std::chrono::steady_clock::now()
                            .time_since_epoch().count());
        b1 = rng() % (M1 - 256) + 257;  // coprime to M1, large alphabet
        b2 = rng() % (M2 - 256) + 257;
        int n = (int)s.size();
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        pw1.assign(n + 1, 1); pw2.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            h1[i+1] = (h1[i]*b1 + (uint64_t)(unsigned char)s[i]) % M1;
            h2[i+1] = (h2[i]*b2 + (uint64_t)(unsigned char)s[i]) % M2;
            pw1[i+1] = (pw1[i]*b1) % M1;
            pw2[i+1] = (pw2[i]*b2) % M2;
        }
    }
    uint64_t operator()(int l, int r) const {
        uint64_t a = (h1[r+1] + M1 - h1[l]*pw1[r-l+1] % M1) % M1;
        uint64_t b = (h2[r+1] + M2 - h2[l]*pw2[r-l+1] % M2) % M2;
        return (a << 32) | b;
    }
};
```

2.3 Bidirectional hashing

For palindromic-substring queries ($S[l..r] = \text{rev}(S[l..r])$?) you need both a forward hash H_f and a reverse hash H_r, where H_r hashes S from right to left. To make them comparable, the reverse hash must use a *different* exponent convention: $\sum S[i] \cdot b^i$ rather than $\sum S[i] \cdot b^{n-1-i}$. The palindrome test is then

$$H_f(S[l..r]) \;=\; b^{r-l+1-1} \cdot H_r(S[l..r]) \cdot b^0 \pmod p$$

after appropriate scaling. Careless exponent alignment is the most common source of bugs in bidirectional hash code; unit-test on a known palindrome like `racecar` before submitting.

2.4 Anti-hash: Thue–Morse in detail

Watch out

The Thue–Morse sequence $T_n = (t_i)$ where t_i = parity of the number of 1s in the binary representation of i is the canonical hash-breaker. For any base b:

- T_{2k} is a Thue–Morse word of length 2^{k+1}.
- Every prefix of T_n of length 2^k has hash collision with some other prefix of length 2^k when n is large enough (roughly $n \geq 2\sqrt{p}$).

Concretely, against a single fixed base b and modulus p, the author precomputes two equal-hash strings of length $2\sqrt{p}$ by enumerating Thue–Morse prefixes. Against a randomised base chosen at runtime, the author cannot do this.

The takeaway: a single hash with randomised base is roughly as safe as a double hash with fixed bases. We prefer double hash + randomised base for the case $qL \approx 10^{12}$ and accept the slight runtime cost.

2.5 Hashing + binary search: longest common substring of k strings

Hybrid pattern

"Longest common substring of k strings $S_1, \ldots, S_k$" (each of length $\leq n$). The hybrid solution: binary search on length L; for each L, compute hashes of all length-L substrings of S_1 and store them in a set; then for $S_2, \ldots, S_k$, intersect the sets with the previous intersection. If the final intersection is non-empty, L is achievable. Total: $\mathcal{O}(k \cdot n \log n)$ with $\mathcal{O}(n)$ memory per pass.

This beats the suffix-tree solution for $k \leq 10$ in implementation simplicity, and ties it asymptotically. The suffix-tree solution is $\mathcal{O}(kn)$ once and lets you answer further queries (e.g. "for each L, how many strings contain it?") — so it wins when queries are interactive.

2.6 When to use hashing, expert edition

When to use

Beyond the primer's list, expert problems solvable by hashing include:

- **Substring comparison queries** answered by hashing + binary search on the LCP length, in $\mathcal{O}(\log n)$ per query. Combined with a sparse table, $\mathcal{O}(1)$ is achievable but the constant is heavier than the SA + sparse table solution.
- **Pattern matching with wildcards** (don't-cares) — use hashing to skip over wildcard-aligned positions, plus FFT for the general case in $\mathcal{O}(n \log n)$.
- **Bidirectional palindrome queries** — H_f and H_r plus a range-LCP binary search.
- **Equality of two concatenations ST vs. TS** without materialising them — hash the components separately and combine with the shift formula.
- **LCP of two suffixes when SAM/SA is unavailable** — hashing + exponential search gives $\mathcal{O}(\log n)$ per query, which is fine when only a few queries are needed.

3 Prefix Function: Automaton View, Periodicity, and Applications

The primer introduced π and KMP search. This chapter extends the picture with the *failure automaton* (the DFA view), the *periodicity lemma* with proof, and three advanced applications: counting distinct factors of X^k, the streaming-KMP variant, and the periodicity-based string DP.

3.1 The failure automaton as a DFA

Definition 3.1 (Failure automaton)**.** Given pattern P, the failure automaton is the DFA with $|P|+1$ states $0,\dots,|P|$ where state i represents "a length-i prefix of P has just been matched". The transition function is

$$\delta(i,c) = \begin{cases} i+1 & \text{if } i < |P| \text{ and } c = P[i], \\ \delta(\pi[i-1],c) & \text{if } i > 0 \text{ and } c \neq P[i], \\ 0 & \text{if } i = 0 \text{ and } c \neq P[0]. \end{cases}$$

The naive recursive definition costs $\mathcal{O}(|P|\cdot|\Sigma|\cdot|P|)$ to build because each $\delta(\pi[i-1],c)$ may recurse. The standard optimisation is to precompute π first, then fill δ row by row using the rule $\delta(i,c) = \delta(\pi[i-1],c)$ when $c \neq P[i]$ (with $\delta(0,c) = 0$ for $c \neq P[0]$). Since $\pi[i-1] < i$, all referenced values are already computed.

Failure automaton, O(m*sigma) build, O(1) transition

```cpp
struct FailureAutomaton {
    int m;
    std::vector<std::array<int, 26>> delta;  // delta[state][c]

    FailureAutomaton(const std::string& pat) : m(pat.size()) {
        std::vector<int> pi(m);
        for (int i = 1; i < m; ++i) {
            int j = pi[i-1];
            while (j > 0 && pat[i] != pat[j]) j = pi[j-1];
            if (pat[i] == pat[j]) ++j;
            pi[i] = j;
        }
        delta.assign(m + 1, {});
        delta[0].fill(0);
        for (int i = 0; i < m; ++i) {
            for (int c = 0; c < 26; ++c) {
                if (c == pat[i] - 'a') delta[i][c] = i + 1;
                else if (i == 0)       delta[i][c] = 0;
                else                   delta[i][c] = delta[pi[i-1]][c];
            }
        }
        // terminal state m: keep its transitions same as state pi[m-1] etc.
        for (int c = 0; c < 26; ++c) delta[m][c] = delta[pi[m-1]][c];
    }
    int advance(int state, char c) const { return delta[state][c - 'a']; }
};
```

3.2 Periodicity lemma, with proof

Theorem 3.2 (Periodicity lemma, weak form)**.** *Let p be a period of S (meaning $S[i] = S[i+p]$ for all i such that $i+p<n$) and let q be another period. If $p+q \le n+\gcd(p,q)$, then S has period $\gcd(p,q)$.*

Proof sketch

The proof is the classical Fine–Wilf argument. Write $g = \gcd(p, q)$ and suppose $p < q$ (the symmetric case follows). For each $i \in [0, n-p-q]$, we have $S[i] = S[i+p] = S[i+p+q] = \ldots$ via the two periodicity assumptions. Working modulo g, the same character appears at every position in the same residue class, so S has period g.

Corollary 3.3 (String powers)**.** *S is a power of a shorter string T (i.e. $S = T^k$ for $k \geq 2$) if and only if $\pi[n-1] = n - |T|$ and $|T|$ divides n. The smallest such T has length $n - \pi[n-1]$.*

This single corollary powers an enormous family of problems:

- "Is S periodic?" — check whether $n - \pi[n-1]$ divides n.
- "Smallest period of S" — $\min\{p : S \text{ has period } p\} = n - \pi[n-1]$ if this divides n, else n.
- "Compress S as T^k" — output $T = S[0..p-1]$, $k = n/p$.
- "Number of cyclic rotations of S that equal S" — equals $\gcd(n, p)$ where p is the smallest period.
- "Can S be tiled by repeating T?" — check $n \bmod |T| = 0$ and the periodicity via π.

3.3 Streaming KMP: the text is a stream

If the pattern is fixed but the text is an arbitrary-length stream, the failure automaton is the only structure that gives $\mathcal{O}(1)$ per character with $\mathcal{O}(|P| \cdot |\Sigma|)$ preprocessing. Aho–Corasick is the multi-pattern generalisation; suffix automaton is the multi-text generalisation. The failure automaton sits at the intersection: single pattern, streaming text.

3.4 Worked example: counting distinct factors of X^k

Let $X = \texttt{aba}$ and consider $S = X^3 = \texttt{abaabaaba}$. Compute π on S:

$$\pi = [0, 0, 1, 1, 2, 3, 4, 5, 6].$$

$\pi[n-1] = 6$, so the smallest period of S is $n - 6 = 3$ (which equals $|X|$). Divided into X^3, S has $|X|$-period structure. To count distinct factors of S: every distinct factor of X^k is already a factor of $X^{k+1} \cdot X^{k+1}$ (two copies are sufficient), so the count is the same as for $S' = \texttt{abaabaabaabaabaaba}$ (six copies). The number of distinct factors is

$$\sum_{i=0}^{n-1} (i+1-\pi[i]) \;=\; (1-0)+(2-0)+(3-1)+(4-1)+(5-2)+(6-3)+(7-4)+(8-5)+(9-6) = 22.$$

A direct enumeration confirms 22 distinct substrings of S.

3.5 When to use the failure automaton

When to use

Beyond the primer's bullets, the failure automaton wins for:

- **Streaming single-pattern matching** with $\mathcal{O}(1)$ per character.
- **String-powers problems** — "is $S = T^k$?", "smallest T", "count cyclic rotations equal to S" — all from $\pi[n-1]$.
- **DP on streaming periodicity** — "for each prefix of the stream, has a period-p substring of length $\geq 2p$ occurred?" Build a sliding automaton that tracks both π and

the period.

- **Boolean convolution via period detection** — if you need to know whether S and T share a length-k factor and k divides both lengths, the period structure of $S+\#+T$ can give the answer without enumeration.

3.6 Codeforces references

- **CF 432D — Prefixes and Suffixes**: count occurrences of each prefix of S that is also a suffix — pure π walking plus a frequency table.
- **CF 126B — Password**: longest substring that is a prefix, a suffix, and occurs in the middle — $\pi[n-1]$ recursion.
- **CF 808G — Homonoia**: DP on periodicity and failure-automaton transitions.

4 Z-Function: Two-Sided Pattern Matching and Wildcards

We assume the reader has read the primer's Z chapter. Here we develop the two applications where Z is genuinely the best tool: *two-sided pattern matching* and *matching with wildcards*.

4.1 Two-sided pattern matching

Suppose we want to find all occurrences of P in T, but with the additional constraint that the match must be consistent with a *split* $P = P_1 \# P_2$ where $\#$ is a wildcard — i.e., the character at the split position in T can be anything. The natural solution is to compute Z on $P_1 \# T$ to find left-side matches, and Z on $\text{rev}(P_2) \# \text{rev}(T)$ to find right-side matches, then combine.

Two-sided pattern matching via Z

```cpp
// Find all positions i in T such that T[i..i+|P1|-1] = P1 and
// T[i+|P1|+1..i+|P|-1] = P2  (the middle character is wildcard).
std::vector<int> twoSided(const std::string& T,
                          const std::string& P1,
                          const std::string& P2) {
    std::string fwd = P1 + '\0' + T;
    auto zf = zFunction(fwd);
    std::string rT(rbegin(T), rend(T));
    std::string rP2(rbegin(P2), rend(P2));
    std::string rev = rP2 + '\0' + rT;
    auto zr = zFunction(rev);
    int n = T.size(), m1 = P1.size(), m2 = P2.size();
    std::vector<int> occ;
    for (int i = 0; i + m1 + 1 + m2 - 1 < n; ++i) {
        int leftMatch  = zf[m1 + 1 + i];               // P1 matches T[i..]
        int rightStart = (n - 1) - (i + m1 + 1 + m2 - 1);
        int rightMatch = zr[m2 + 1 + rightStart];      // P2 matches
        if (leftMatch >= m1 && rightMatch >= m2) occ.push_back(i);
    }
    return occ;
}
```

This pattern is the heart of the Galil–Seiferas algorithm for k-error matching on small patterns. It is also the conceptual ancestor of the bit-parallel k-error algorithm we will see in Chapter 12.

4.2 Matching with wildcards via Z (single wildcard)

For a single wildcard position, the two-sided approach generalises naturally: split the pattern at the wildcard and use Z on each side. For multiple wildcards, the trick is to split the pattern at the longest non-wildcard run and apply two-sided matching, then verify the remaining positions by hashing.

4.3 When to use Z, expert edition

When to use

Beyond the primer's bullets, Z wins for:

- **Two-sided matching** (single-pattern matching with one or a few wildcard positions, often from a problem's natural split structure).
- **Streaming periodicity detection** when the period is fixed: Z on the prefix $S[0..k-1]$ for the running prefix gives the period structure in $\mathcal{O}(k)$ per step.
- **LCP-based deduplication** of suffix sets when you cannot afford a full suffix array (e.g. small $n \log^2 n$ beats $5n$ in memory-tight contests).
- **Palindrome test with double concatenation** — concatenate $S + \# + \mathrm{rev}(S)$ and look for $z[i] = n$ at the appropriate position.

4.4 Codeforces references

- **CF 1051E — Vasya and Big Integers**: DP with hashing + Z for prefix/suffix tests.
- **CF 1090K**: Z-based approach for common-prefix queries on a class of strings.

5 Trie and Aho–Corasick: Matrix Exponentiation and Counting

The primer covered Aho–Corasick's failure links and matching. The expert twist is to treat the Aho–Corasick automaton as a *linear recurrence* over its transition graph: count the number of strings of length n that match (or avoid) the patterns, using matrix exponentiation. This is the canonical hybrid pattern "AC + matrix".

5.1 Reformulation: AC automaton as a DFA

After building the goto-shortcut version of the AC automaton, the structure is a deterministic finite automaton with $|Q| \le \sum |P_i| + 1$ states and $|\Sigma|$ transitions per state. Some states are *accepting* (a pattern ends there, directly or via an output link). The DFA accepts exactly the strings over Σ^* that contain at least one pattern as a substring.

Definition 5.1 (AC adjacency matrix)**.** Let A be the $|Q| \times |Q|$ matrix where $A[u][v] =$ the number of characters $c \in \Sigma$ such that $\delta(u, c) = v$. Then the entry $(A^n)[0][v]$ counts the number of strings of length n that lead from the start state to state v. The total number of strings of length n that contain at least one pattern is

$$\sum_{v \text{ accepting}} (A^n)[0][v].$$

The matrix exponentiation is $\mathcal{O}(|Q|^3 \log n)$. With $|Q| \le 100$ and $n \le 10^{18}$, this is well under one second.

5.2 Implementation

AC matrix exponentiation for counting strings avoiding patterns

```cpp
// Count strings of length n over Sigma = {a..z} that AVOID every pattern.
// 1. Build AC automaton; mark "banned" states (state is accepting if it
//    represents a string ending in any banned pattern, including via
//    output links).
// 2. Form adjacency matrix A (only transitions to non-banned states).
// 3. Compute sum_{v safe} (A^n)[0][v] via fast matrix exponentiation.
typedef std::vector<std::vector<long long>> Matrix;
const long long MOD = 1'000'000'007LL;

Matrix matMul(const Matrix& A, const Matrix& B) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Matrix C(n, std::vector<long long>(m, 0));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j) {
            long long s = 0;
            for (int t = 0; t < k; ++t)
                s = (s + A[i][t] * B[t][j]) % MOD;
            C[i][j] = s;
        }
    return C;
}

Matrix matPow(Matrix A, long long e) {
    int n = A.size();
    Matrix R(n, std::vector<long long>(n, 0));
    for (int i = 0; i < n; ++i) R[i][i] = 1; // identity
    while (e > 0) {
        if (e & 1) R = matMul(R, A);
        A = matMul(A, A);
        e >>= 1;
    }
    return R;
}
```

The complete flow: build the AC automaton with array transitions and goto-shortcuts; propagate the "banned" flag down the output-link chain (a state is banned if any pattern ends there, directly or indirectly); remove all transitions into banned states from the adjacency matrix; exponentiate.

5.3 Worked example

Patterns $\{\texttt{ab}, \texttt{ba}\}$, alphabet $\Sigma = \{a, b\}$, $n = 4$. The AC automaton has 5 states: 0 (root), 1 (`a`), 2 (`b`), 3 (`ab`, banned), 4 (`ba`, banned). The banned-propagation makes state 3 and 4 unreachable for further counting (their out-transitions don't matter, but we drop them). The remaining safe adjacency matrix is 3×3:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \text{transitions: } 0 \xrightarrow{a} 1, 0 \xrightarrow{b} 2, 1 \xrightarrow{a} 1, 2 \xrightarrow{b} 2.$$

Computing $\sum_{v \text{ safe}} (A^4)[0][v]$: the safe states are $0, 1, 2$, and we count strings of length 4 that never hit a banned state. Such strings have no `ab` and no `ba`, so they are all a's or all b's: `aaaa`, `bbbb`, giving 2. The matrix computation agrees.

5.4 When to use AC + matrix exponentiation

When to use

Reach for AC + matrix when:

- The problem asks "how many strings of length n over Σ contain/avoid any of the patterns $P_1, \ldots, P_k$?", with n up to 10^{18}.
- "The count modulo p" is asked, indicating the answer is huge.
- "After walking n steps, what is the probability of being in state v?" — Markov-chain-on-AC, the same matrix machinery.

The hard limit is $|Q| \leq 200$ for matrix exponentiation to fit in time ($|Q|^3 = 8 \times 10^6$ per multiplication, $\log n \approx 60$ gives $\sim 5 \times 10^8$ operations).

5.5 Codeforces references

- **CF 86C — Magical String**: AC + matrix exponentiation for counting strings that contain a given pattern.
- **CF 966B — Kniv** : smaller AC + DP variant.
- **CF 1202E**: classic AC + DP, no matrix exponentiation needed (the DP is $\mathcal{O}(n \cdot |Q|)$).

6 Suffix Array: SA-IS, DC3, and Sparse-Table RMQ

The primer's $\mathcal{O}(n \log^2 n)$ suffix array via sort-by-prefix is enough for $n \leq 10^6$. For $n \geq 10^7$ (genome-scale problems, IOI training) the constant factor becomes unacceptable, and we need the linear-time SA-IS or DC3 algorithm. We give a conceptual treatment of SA-IS, the linear-time LCP via Kasai (with the proof that the loop is amortised $\mathcal{O}(n)$), and the sparse-table RMQ.

6.1 SA-IS in concept

SA-IS (Suffix Array Induced Sorting) by Nong, Zhang, and Chan (2009) is the standard linear-time SA construction. The idea is recursive:

1. Classify each suffix as S-type (if it is lexicographically smaller than its right neighbour) or L-type (otherwise). Equivalently, $S[i]$ is S-type iff $S[i..] < S[i+1..]$.
2. An LS-position is an L-type position i such that $i+1$ is S-type; these are the "key positions" for induced sorting.
3. Sort the LS-suffixes recursively by reducing the alphabet via renaming.
4. Induce-sort the rest: first the S-type suffixes from the LS-sorted ones, then the L-type suffixes from the S-sorted ones. The induction is by a left-to-right then right-to-left scan over the bucket array.

Note

A full implementation of SA-IS runs 80–120 lines of C++ and is well worth having in your template. For most CP problems, the $\mathcal{O}(n \log^2 n)$ version is enough; SA-IS is for $\geq 10^7$ inputs or when memory is tight (the recursive version uses $\sim 4n$ bytes plus the recursion stack).

6.2 Linear LCP: Kasai's algorithm, with the amortisation proof

Theorem 6.1 (Kasai's lemma)**.** *Let* rank$[i]$ *be the rank of suffix* $S[i..]$ *in the sorted suffix array. Define* $h_i = \text{lcp}(S[i..], S[\text{sa}[\text{rank}[i]-1]..])$*. Then* $h_{i+1} \geq \max(0, h_i - 1)$.

Proof sketch

Consider the suffix $S[i..]$ and its predecessor sa$[\text{rank}[i]-1]$. If we drop the first character from each, we get $S[i+1..]$ and $S[\text{sa}[\text{rank}[i]-1]+1..]$, which share an LCP of length $h_i - 1$ (or 0 if $h_i = 0$). But $S[\text{sa}[\text{rank}[i]-1]+1..]$ is some suffix; the LCP of $S[i+1..]$ with its actual predecessor in the SA is at least the LCP with any other suffix, so $h_{i+1} \geq h_i - 1$.

This lemma means the loop counter h can be decremented by at most 1 between consecutive iterations, so the total work is $\mathcal{O}(n)$.

6.3 Sparse table for RMQ on LCP, with build complexity

The sparse table over the LCP array lets us answer "lcp$(S[i..], S[j..])$" in $\mathcal{O}(1)$ for any $i \neq j$:

1. Translate i, j into ranks: $r_i = \text{rank}[i]$, $r_j = \text{rank}[j]$.
2. Compute $l = \min(r_i, r_j)$, $r = \max(r_i, r_j) - 1$ (the LCP array is indexed by sorted rank).
3. Answer: $\min\{\text{lcp}[k] : l \leq k \leq r\}$, computed by the sparse table in $\mathcal{O}(1)$.

The build is $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n \log n)$ memory. For $n = 10^6$, this is $\sim 20 \cdot 10^6 \cdot 4 = 80$ MB, uncomfortable but legal.

6.4 Distinct substrings, with proof

Theorem 6.2 (Distinct substrings count)**.** *The number of distinct substrings of* S *is* $\frac{n(n+1)}{2} - \sum_{i=1}^{n-1} \text{lcp}[i]$.

Proof sketch

Total substrings $= \binom{n+1}{2}$. Each pair of adjacent suffixes in the SA shares exactly lcp$[i]$ prefixes. Because the SA is sorted, the LCP of any two non-adjacent suffixes equals the minimum of the LCPs in the interval — so any two suffixes sharing a prefix of length k have all k prefixes double-counted. Summing the adjacent LCPs gives exactly the total overcount.

6.5 Smallest rotation via SA in $\mathcal{O}(n \log n)$

Concatenate $S + S$, build the suffix array, and find the smallest suffix whose starting index is in $[0, n-1]$. The starting index gives the rotation. This is asymptotically slower than Booth's $\mathcal{O}(n)$ but uses the same machinery as the rest of the SA pipeline, so it is often easier to code in-contest.

6.6 When to use SA, expert edition

When to use

Expert problems where SA + LCP + sparse table is the right answer:

- **Arbitrary substring comparison** in $\mathcal{O}(1)$ after preprocessing.
- **Longest common substring of k strings**, $k \geq 2$ — concatenate with sentinels, build SA, sliding-window minimum over LCP across k different source strings.
- **k-th lexicographically smallest substring of S** — binary search on the suffix

array.

- **Number of occurrences of each substring** — use the suffix-link tree structure (each suffix corresponds to a path in the SA + LCP).
- **Pattern matching with multiple short patterns** — binary search each pattern in $\mathcal{O}(|P| \log n)$; saves the cost of building AC if the patterns are not given upfront.

6.7 Codeforces references

- **CF 1066F**: SA + LCP for substring queries.
- **CF 1238D — AB-string**: SA-based enumeration of substring covers.
- **CF 427D — Match & Catch**: SA + LCP for two-string longest common substring.
- **CF 1090J — Two Paths**: SA + LCP for paths in a graph with edge labels.

7 Suffix Automaton: Endpos Tree, Multi-String, and Substring DP

The primer introduced the suffix automaton (SAM) and its online construction. This chapter develops two expert extensions: the *suffix-link tree* as a DP substrate, and the *generalised SAM* for multiple strings.

7.1 The suffix-link tree

Definition 7.1 (Suffix-link tree)**.** The suffix links $\text{link}(v)$ of every SAM state form a tree rooted at the empty-string state. This tree is called the *suffix-link tree* (or *parent tree*); it has at most $2n - 1$ nodes and $2n - 2$ edges.

The crucial structural fact is that $\text{endpos}(v) \subsetneq \text{endpos}(\text{link}(v))$ — a state's endpos set is a strict subset of its parent's. So the suffix-link tree partitions the suffixes of S by their endpos-equivalence classes. Subtree sums give endpos sizes (occurrence counts); subtree paths give suffix inheritance.

7.2 DP on the suffix-link tree

The standard DP: process states in decreasing order of $\text{len}(v)$, propagate $\text{cnt}(v)$ up the link tree:

$$\text{cnt}(\text{link}(v)) \; += \text{cnt}(v).$$

After this single propagation pass, $\text{cnt}(v)$ is $|\text{endpos}(v)|$ — the number of occurrences of any substring in v's equivalence class.

Suffix-link tree DP for occurrence counts

```
void computeOccurrences(SuffixAutomaton& sam) {
    // cnt[v] = 1 for non-clone states, 0 for clones (set during extension).
    int n = (int)sam.st.size();
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int a, int b) {
        return sam.st[a].len > sam.st[b].len;
    });
    for (int v : order) {
        if (sam.st[v].link != -1)
            sam.st[sam.st[v].link].cnt += sam.st[v].cnt;
    }
```

```
}
```

7.3 Number of distinct substrings ending at each prefix

After processing the k-th character of S (so we have just inserted $S[k]$), the number of new distinct substrings created is $\text{len}(\text{cur}) - \text{len}(\text{link}(\text{cur}))$. Summing this over the entire construction gives the total distinct substrings — a fast alternative to the SA-based count when the input is online.

7.4 Generalised suffix automaton

To build a SAM for multiple strings $S_1, \ldots, S_k$, we generalise: reset `last` to the root between strings, but keep the same automaton. The catch is that when extending with c, we may have to walk up the suffix-link chain *across the previously built parts* of the automaton — the existing transitions for c from a state must be checked for length-consistency, exactly as in the single-string case but with a stricter check.

Generalised SAM extension (sketch)

```
void extendGeneralised(SuffixAutomaton& sam, char c) {
    int last = sam.last;
    int cur = sam.extend(c);          // standard extend; returns new state index
    // After extension, if the automaton already had a transition from the
    // previous last for c, sam.st[cur].link may have been set to a state whose
    // len is too large; we need to clone, exactly as in the single-string case.
    // (The standard SAM code already handles this correctly, but only when
    // `last` is the actual last-insertion state of the current string. The
    // trick is: between strings, set `sam.last = 0` (root) to re-anchor.)
}
// Between two strings, do: sam.last = 0;
```

The applications of generalised SAM:

- "For each T, how many substrings of S does it contain?" — build SAM on S, walk through T.
- "Common substrings of $S_1, \ldots, S_k$" — build generalised SAM, mark each state with which strings have a substring in that class; states marked with all k strings contribute their len − len(link) to the LCS count.

7.5 Longest common substring of two strings, in $\mathcal{O}(|S_1| + |S_2|)$

Build SAM on S_1. Walk through S_2 tracking (current state v, current matched length l). For each character c of S_2:

- If v has a transition on c: take it, $l \mathrel{+}= 1$.
- Else while $v \neq$ root and no transition on c: $v = \text{link}(v)$, $l = \text{len}(v)$. If now v has the transition, take it and $l = \text{len}(v) + 1$; else $v = \text{root}, l = 0$.

The answer is $\max l$ seen during the walk.

7.6 When to use SAM, expert edition

When to use

Expert SAM applications:

- **Online substring queries** (the killer feature) — each new character takes $\mathcal{O}(1)$ amortised to update.
- **Longest common substring of two strings** in linear time.
- **Cyclic-shift queries** — build SAM on S, walk from root picking smallest transition, n times.
- **Substring-count DP** over the suffix-link tree (occurrence counts).
- **Distinct substrings count** online, 1 unit per character.
- **Generalised SAM for multi-string problems** — build, then per-state bitmask of which strings have substrings in that class.
- **SAM + DP** on substrings with frequency constraints.

7.7 Codeforces references

- **CF 235C — Cyclical Quest**: the canonical SAM problem.
- **CF 204E — Little Katy and the Suffixes**: SAM + suffix-link tree DP for multi-string occurrence counts.
- **CF 432D — Prefixes and Suffixes**: SAM-based suffix-count approach.
- **CF 700B**: suffix-link tree DP substrate.

8 Ukkonen's Online Suffix Tree

The suffix tree is the historical ancestor of the suffix automaton. Where the SAM compresses by endpos equivalence, the suffix tree explicitly spells out every suffix as a path from the root, using *implicit edges* (a character plus a length, rather than the full substring) to keep the size linear. Ukkonen's algorithm builds it online in $\mathcal{O}(n)$ time.

8.1 Why learn the suffix tree?

If you know SAM, why bother with the suffix tree? Three reasons:

1. Some problems are more natural on the tree: "find every occurrence of T as a substring of S and report them in sorted order of starting position" — the suffix tree gives this directly via a single subtree; the SAM needs an extra DFS over the link tree.
2. The suffix tree's LCA structure gives $\mathcal{O}(1)$ LCP queries via Euler-tour + sparse table, after $\mathcal{O}(n)$ preprocessing. This is the basis of the LCP array construction algorithm we met in Chapter 6.
3. The suffix tree is the substrate of the Burrows–Wheeler transform (Chapter 10); the BWT array is the suffix tree's leaves in DFS order.

8.2 Implicit edges and the active point

Definition 8.1 (Suffix tree)**.** The *suffix tree* of S (terminated with a unique sentinel \$) is the compacted trie of all suffixes of S\$. Each edge is labelled by a substring of S\$, and edges are

stored implicitly as pairs $(\text{start}, \text{length})$ into S. The tree has at most $2n$ nodes and $2n - 1$ edges, and can be built in $\mathcal{O}(n)$ time online.

Ukkonen's algorithm maintains an *active point*:

- active_node: the node we are currently extending from.
- active_edge: the first character of the edge we are descending into (or $\perp$).
- active_length: how far down that edge we are.

At each step, we extend the suffix tree by inserting the suffix $S[i..]$ for each i from the previous endpoint to the current position. The clever part: most of these insertions are *implicit* (Rule 3 below), and the active point advances lazily, so we do at most one real extension per character.

8.3 The three suffix extension rules

1. **Rule 1 (new leaf)**: If the active point is at a node and there is no edge starting with $S[k]$, create a new leaf edge from that node labelled $S[k..]$.
2. **Rule 2 (split edge)**: If the active point is on an edge and the next character differs from $S[k]$, split the edge: insert an internal node at the active point, and add a new leaf from that node with edge label $S[k..]$.
3. **Rule 3 (already present)**: If the next character matches $S[k]$, do nothing — just advance the active length by 1.

After applying any rule, if we are not at the root, we follow a *suffix link* from the active node to advance to the next suffix position. Suffix links are added when an internal node is created: it points to the previous internal node created in the same phase.

8.4 Amortised linear time

Three tricks make Ukkonen $\mathcal{O}(n)$:

1. **Edge labels as pairs**: storing $S[l..r]$ as (l, r) means edge length is $\mathcal{O}(1)$, not $\mathcal{O}(n)$.
2. **Implicit extensions (Rule 3)**: skip already-present suffixes without walking; just advance the active length.
3. **Suffix links**: jumping to the next suffix via a single pointer avoids restarting from the root.

The amortised analysis bounds the total work across all phases by $\mathcal{O}(n)$: each character is added at most twice across the whole construction (once as an explicit Rule 1/2 insert, once as a Rule 3 implicit extension).

8.5 Implementation sketch

A full Ukkonen implementation is 200–300 lines of C++ and is rarely needed in CP — the suffix array is shorter and answers the same queries. The exception is the *online* setting where queries arrive interleaved with character additions; in that case Ukkonen is the only linear-time option.

Ukkonen suffix tree, skeletal (refer to cp-algorithms for full)

```cpp
struct SuffixTree {
    struct Node {
        int start, *end;           // edge label S[start..end] (inclusive)
        int link = -1;              // suffix link
        std::map<char, int> next;
    };
    std::string s;
    std::vector<Node> st;
    int activeNode = 0, activeEdge = -1, activeLen = 0;
```

```
    int remaining = 0;              // suffixes to add this phase
    int leafEnd = 0;                // shared end pointer for all leaves

    void extend(char c) {
        s.push_back(c);
        leafEnd = (int)s.size() - 1;
        ++remaining;
        int lastNew = -1;
        while (remaining > 0) {
            if (activeLen == 0) {
                if (st[activeNode].next.count(c)) {
                    activeEdge = st[activeNode].next[c];
                    activeLen = 1;
                    break; // Rule 3
                } else {
                    int leaf = newLeaf((int)s.size() - 1, &leafEnd);
                    st[activeNode].next[c] = leaf;
                    --remaining;
                    if (activeNode != 0) activeNode = st[activeNode].link;
                }
            } else {
                // active point is on an edge; check next char
                // ... (split or advance, see cp-algorithms.com/string/suffix-tree-ukkonen)
            }
        }
    }
    int newLeaf(int start, int* endPtr) {
        Node n; n.start = start; n.end = endPtr; n.link = -1;
        st.push_back(n); return (int)st.size() - 1;
    }
};
```

8.6 Applications

- **Longest repeated substring** = depth of the deepest internal node.
- **Longest common substring of k strings** = deepest node whose subtree contains leaves from all k strings (generalised suffix tree).
- **All occurrences of P in S** = walk the suffix tree from the root following P; if you arrive at a node v, all occurrences are the starting positions of leaves in v's subtree.
- **LCA queries on suffixes** via Euler-tour + sparse-table: $\mathcal{O}(1)$ LCP of any two suffixes.
- **BWT (Chapter 10)** is the leaf sequence in DFS order.

8.7 When to use the suffix tree

When to use

Use Ukkonen's suffix tree when:

- The problem is fundamentally online and asks substring queries interleaved with character additions.
- You need *sorted* occurrence lists (subtree DFS gives them naturally).
- The problem reduces to LCA on suffixes (then Euler-tour + sparse table on the suffix tree).
- The problem is the construction of BWT or FM-index (next chapter).

In practice, SAM + suffix-link tree DP replaces 90% of suffix-tree applications at lower memory cost. Reserve Ukkonen for problems that really need its online structure or its LCA substrate.

8.8 Codeforces references

- **CF 235C — Cyclical Quest**: SAM is simpler, but the suffix-tree approach with cyclic shifts is also standard.
- **CF 427D — Match & Catch**: suffix tree (or SAM) on two concatenated strings.
- **Emax blog**: theoretical reference for the suffix tree.

9 Generalised Suffix Automaton for Multiple Strings

The generalised suffix automaton (GSAM) extends the SAM to a *set* of strings $S_1, \ldots, S_k$. Its states correspond to endpos-equivalence classes that are restricted to the per-string endpos sets, allowing multi-string substring queries.

9.1 Construction

The construction is identical to single-string SAM with one twist: between two strings, reset `last` to the root. Each call to `extend(c)` must check the length condition carefully because the state link(cur) points to may belong to a different string's portion of the automaton.

Generalised SAM, multi-string

```
struct GeneralisedSAM {
    struct State {
        int len = 0, link = -1;
        std::map<char, int> next;
        // mask[i] = bit i set if string S_i has a substring in this class
        std::vector<uint64_t> mask; // size = (k+63)/64
    };
    std::vector<State> st = {State{}};
    int last = 0;

    void extend(char c, int stringIdx) {
        int cur = (int)st.size();
        st.push_back({st[last].len + 1, -1, {}, {}});
        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = (int)st.size();
                st.push_back({st[p].len + 1, st[q].link, st[q].next, st[q].mask});
                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
```

```
            }
        }
        // mark the new state's class for this string
        ensureMaskSize(cur, stringIdx);
        st[cur].mask[stringIdx / 64] |= (1ULL << (stringIdx % 64));
        last = cur;
    }
    void newString() { last = 0; }
    void ensureMaskSize(int state, int stringIdx) {
        if ((int)st[state].mask.size() * 64 <= stringIdx)
            st[state].mask.resize((stringIdx / 64) + 1, 0);
    }
};
```

After the build, propagate the bitmask up the suffix-link tree (each parent gets its children's bits OR'd in). A state is "in all k strings" iff its mask has all k bits set.

9.2 Applications

- **Longest common substring of k strings** — find the deepest state with full mask; $\mathcal{O}(\sum |S_i|)$ after the link-tree propagation.
- **For each substring T, which of the k strings contain it?** — walk the GSAM following T; the result state's mask is the answer.
- **Number of substrings common to all k strings** — sum $\text{len}(v) - \text{len}(\text{link}(v))$ over states v with full mask.

9.3 When to use the generalised SAM

When to use

Use GSAM when the problem involves ≥ 2 strings and asks about *common* or *per-string* substring counts. For 2 strings, the suffix tree (generalised) or a simpler SAM-on-S_1-walk-S_2 approach may suffice; for $k \geq 3$ the GSAM is the only linear-time structure.

10 Burrows–Wheeler Transform and FM-index

The Burrows–Wheeler transform (BWT) is a permutation of S that is reversible and clusters equal characters together (so it compresses well). The FM-index (Ferragina–Manzini, 2000) is the corresponding compressed full-text index, supporting pattern matching in $\mathcal{O}(|P| \log \sigma)$ time using $\mathcal{O}(n)$ bits of memory at one byte per symbol. Together they underpin `bzip2` and `bwa` (the standard DNA aligner).

10.1 Definition of BWT

Definition 10.1 (BWT)**.** Sort all cyclic rotations of $S\$$ (the input terminated with a sentinel \$ that is smaller than every character). The BWT is the last column of the resulting matrix: $\text{BWT}[i] =$ the last character of the i-th sorted rotation. Equivalently, $\text{BWT}[i] = S[\text{sa}[i] - 1 \bmod n]$, where sa is the suffix array of $S\$$.

10.2 Reversibility: the LF mapping

The BWT is invertible because of the *LF-mapping*: a bijection $\text{LF} : [0, n) \to [0, n)$ where $\text{LF}(i)$ is the row of the sorted matrix that contains the same character as row i, but where it appears in the

first column. Concretely, if $\text{BWT}[i] = c$ and i is the k-th occurrence of c in the BWT, then $\text{LF}(i)$ is the row of the first column where c appears for the k-th time. This is computed in $\mathcal{O}(1)$ given two auxiliary arrays: $C[c]$ = the total count of characters $< c$ in S, and $\text{Occ}(c, i)$ = the number of times c appears in $\text{BWT}[0..i-1]$.

10.3 Construction

For practical CP purposes, build the BWT from the suffix array of $S\$$: $\text{BWT}[i] = S[\text{sa}[i]-1]$ if $\text{sa}[i] > 0$, else \$.

BWT via suffix array

```
std::string bwt(const std::string& s_in) {
    std::string s = s_in + '\1';  // sentinel '\1' < every printable char
    std::vector<int> sa = buildSA(s);  // O(n log^2 n) from primer
    std::string bwt;
    bwt.reserve(s.size());
    for (int i = 0; i < (int)s.size(); ++i) {
        if (sa[i] == 0) bwt += '\1';                  // pseudo-sentinel
        else            bwt += s[sa[i] - 1];
    }
    return bwt;
}
```

10.4 FM-index: count and locate queries

Given the BWT and the LF-mapping, the FM-index supports:

- count(P): the number of occurrences of P in S. Process P from right to left; at each step, maintain an interval $[lo, hi)$ of BWT rows; the next interval is $[C[c] + \text{Occ}(c, lo), C[c] + \text{Occ}(c, hi))$. The final $hi - lo$ is the count.
- locate(P): enumerate occurrences. Repeatedly apply LF to walk back to the start of S, counting steps; for large outputs, use a sampled suffix-array checkpoint.

FM-index count

```
struct FMIndex {
    std::string bwt;
    int n, sigma = 256;
    std::vector<int> C;        // C[c] = # of chars < c in S
    std::vector<std::vector<int>> occ;  // occ[c][i] = # of c in bwt[0..i-1]

    FMIndex(const std::string& s_in) {
        bwt = bwtFromSuffixArray(s_in);
        n = (int)bwt.size();
        // build C
        std::vector<int> cnt(256, 0);
        for (char c : bwt) cnt[(unsigned char)c]++;
        C.assign(256, 0);
        for (int c = 1; c < 256; ++c) C[c] = C[c-1] + cnt[c-1];
        // build occ table (n log n memory; use wavelet tree for log n)
        occ.assign(256, std::vector<int>(n + 1, 0));
        for (int i = 0; i < n; ++i) {
            for (int c = 0; c < 256; ++c) occ[c][i+1] = occ[c][i];
            ++occ[(unsigned char)bwt[i]][i+1];
        }
    }
    // Count occurrences of P in S. Returns (lo, hi); hi - lo is the count.
    std::pair<int,int> count(const std::string& P) const {
        int lo = 0, hi = n;
```

```
        for (int i = (int)P.size() - 1; i >= 0 && lo < hi; --i) {
            unsigned char c = (unsigned char)P[i];
            lo = C[c] + occ[c][lo];
            hi = C[c] + occ[c][hi];
        }
        return {lo, hi};
    }
};
```

10.5 Wavelet tree: the compressed Occ table

The naive Occ table uses $n\sigma$ integers, which is ~ 250 GB for a 10^6-character text. The wavelet tree reduces this to $n \log \sigma$ bits via a binary-tree structure on the alphabet: each tree node holds a bit-vector indicating whether each character is in the left or right child, and queries walk the tree. With the wavelet tree, $\text{Occ}(c, i)$ is $\mathcal{O}(\log \sigma)$, making the FM-index truly $\mathcal{O}(n)$ bits.

10.6 When to use BWT / FM-index

When to use

Use BWT / FM-index when:

- The text is static but large ($n \geq 10^7$) and pattern queries are short.
- Memory is the bottleneck — the FM-index is $\sim 1.5n$ bytes including the wavelet tree, vs $4n$ bytes for a basic SA + sparse table.
- The problem requires *compressed* full-text search.
- You are solving a bioinformatics problem (DNA reads).

Avoid BWT/FM in CP if $n \leq 10^6$ — the SA pipeline is shorter to code and answers the same queries faster.

10.7 Codeforces references

- **CF 1200F — Graph Travel**: BWT-like graph traversal structure.
- **Emax blog on suffix structures**: includes BWT in the broader context.

11 Bit-Parallel String Algorithms

When the pattern fits in a machine word ($|P| \leq 64$), the operations of KMP, Z, and edit distance can be parallelised across all positions of the text simultaneously using bit-parallelism. The speed-up over the standard algorithms is roughly $w = 64$.

11.1 Shift-And / Shift-Or exact matching

The Shift-And algorithm maintains a bit-vector D where $D[i] = 1$ iff the prefix $P[0..i]$ has just been matched. Reading a new text character c updates D as

$$D' \leftarrow ((D \ll 1) \mid 1) \,\&\, M[c],$$

where $M[c]$ is the bit-mask of positions in P where the character equals c. A match ends at the current text position iff $D' \& (1 \ll (|P| - 1)) \neq 0$. The total time is $\mathcal{O}(n + |P| \cdot \sigma/w)$ to build the masks and $\mathcal{O}(n \cdot |P|/w)$ to scan — effectively $\mathcal{O}(n)$ for $|P| \leq 64$.

Shift-Or exact matching, 64-bit

```cpp
#include <cstdint>
std::vector<int> shiftOr(const std::string& T, const std::string& P) {
    int m = (int)P.size();
    if (m == 0 || m > 64) return {};  // use KMP for longer patterns
    uint64_t mask[256] = {};
    for (int i = 0; i < m; ++i)
        mask[(unsigned char)P[i]] |= (1ULL << i);
    // Shift-Or uses inverted bits: state bit 1 = "not matched"
    uint64_t state = ~0ULL;
    uint64_t goal = 1ULL << (m - 1);
    std::vector<int> occ;
    for (int i = 0; i < (int)T.size(); ++i) {
        state = ((state << 1) | ~mask[(unsigned char)T[i]]);
        // match if bit m-1 is 0 (i.e. NOT set)
        if (~(state >> (m-1)) & 1) occ.push_back(i - m + 1);
    }
    return occ;
}
```

11.2 BNDM: backward nondeterministic matching

BNDM (Backward Nondeterministic Dawg Matching) is the bit-parallel version of the Boyer–Moore algorithm. It scans the text in windows of length $|P|$, processing each window from right to left and using bit-vectors to maintain the set of viable prefixes. On average, BNDM shifts the window by more than one character per text comparison, so it is sublinear on random text. On adversarial text (e.g., $T = a^n$, $P = a^{30}b$), it degrades to $\mathcal{O}(n)$.

11.3 Myers' bit-vector edit distance

Theorem 11.1 (Myers 1999)**.** *The edit distance of A and B (with $|B| \le w$) can be computed in $\mathcal{O}(|A| \cdot |B|/w) = \mathcal{O}(|A|)$ time, using $\mathcal{O}(|B|)$ auxiliary bit-vectors.*

The trick is to maintain, for each row of the standard DP table, four bit-vectors representing the horizontal, vertical, and diagonal differences ($+1, -1, 0$ carry propagation). Each DP row update is a constant number of bit operations (AND, OR, XOR, shift, NOT) regardless of row width, hence w cells per cycle.

Myers bit-vector edit distance (skeletal)

```cpp
// Computes edit distance between A (long) and B (short, |B| <= 64).
// Returns the distance. The full code is ~30 lines; see Myers (1999)
// or the implementation in edlib for a production-grade version.
int myersEditDistance(const std::string& A, const std::string& B) {
    int m = (int)B.size();
    uint64_t Peq[256] = {};
    for (int i = 0; i < m; ++i) Peq[(unsigned char)B[i]] |= (1ULL << i);

    uint64_t Pv = ~0ULL;       // positive vertical delta (init: all 1)
    uint64_t Mv = 0;             // negative vertical delta (init: all 0)
    uint64_t Score = m;          // current edit distance

    uint64_t ones = (m == 64) ? ~0ULL : ((1ULL << m) - 1);
    Pv &= ones;  // mask off bits beyond column m

    for (char c : A) {
        uint64_t Eq = Peq[(unsigned char)c];
        uint64_t Xv = Eq | Mv;
        uint64_t Xh = (((Eq & Pv) + Pv) ^ Pv) | Eq | Mv;
```

```
        uint64_t Ph = Xv & ~Xh;
        uint64_t Mh = Xh & ~Xv;
        if (Ph & (1ULL << (m-1))) ++Score;
        else if (Mh & (1ULL << (m-1))) --Score;
        Ph = (Ph << 1) | 1;
        Mh = (Mh << 1);
        Pv = Mh | ~(Xv | Ph);
        Mv = Pv & Xv;
        Pv = Pv & ones;
        Mv = Mv & ones;
    }
    return (int)Score;
}
```

11.4 When to use bit-parallel

When to use

Reach for bit-parallel when:

- The pattern is short (≤ 64) and the text is long — Shift-And/Or and BNDM are simple to write and fast.
- You need edit distance / LCS where one string is ≤ 64 characters — Myers' bit-vector is $\mathcal{O}(n)$ instead of $\mathcal{O}(nm)$.
- You need k-error matching (next chapter) with small k.
- You need pattern matching with character classes / wildcards (each character's mask is an OR of class members).

Avoid bit-parallel if $|P| > 64$ — the algorithms fall back to multi-word, which loses the speed advantage and adds bookkeeping.

11.5 Codeforces references

- **CF 5E — Subblock**: classic DP where bit-parallel gives $\mathcal{O}(n^2/w)$ vs the naive $\mathcal{O}(n^3)$.
- **CF 514C — Watto and Mechanism**: trie + 1-error matching; bit-parallel variant for small alphabet.

12 Approximate Pattern Matching with k Errors

Approximate matching finds all positions in T where some substring of length $|P|$ differs from P in at most k characters (Hamming distance) or has edit distance at most k (Levenshtein distance). This is a workhorse in bioinformatics (DNA read alignment) and a regular CP problem.

12.1 Hamming distance k-mismatch

For Hamming distance (substitutions only, no insertions/deletions), the standard DP is $\mathcal{O}(|T|\cdot|P|)$. For small k there are two faster approaches:

1. **Landau–Vishkin** $\mathcal{O}(|T| \cdot k)$: use the diagonal-and-anti-diagonal structure of the DP table; only the cells within k of each diagonal matter.
2. **Bit-parallel Hamming** $\mathcal{O}(|T| \cdot |P|/w)$: maintain, for each $j \in 0..k$, a bit-vector R_j where $R_j[i] = 1$ iff $P[0..i]$ matches the current text window with at most j mismatches. Update R_{j+1} from R_j by OR-ing in the new mismatch positions.

Bit-parallel k-mismatch (Hamming)

```cpp
// Find all positions i in T such that Hamming(T[i..i+|P|-1], P) <= k.
// Pattern length must be <= 64.
std::vector<int> kMismatch(const std::string& T,
                           const std::string& P, int k) {
    int m = (int)P.size();
    if (m > 64) return {};
    uint64_t mask[256] = {};
    for (int i = 0; i < m; ++i)
        mask[(unsigned char)P[i]] |= (1ULL << i);
    // R[j] = bit-vector of prefixes P[0..i] with <= j mismatches so far
    std::vector<uint64_t> R(k + 1);
    for (int j = 0; j <= k; ++j) R[j] = ~0ULL;   // initially, all prefixes are 0-mismatch
    with empty
    std::vector<int> occ;
    for (int t = 0; t < (int)T.size(); ++t) {
        uint64_t prev = R[0];
        R[0] = (R[0] << 1) | mask[(unsigned char)T[t]];
        for (int j = 1; j <= k; ++j) {
            uint64_t newR = ((R[j] << 1) | mask[(unsigned char)T[t]])
                          & (prev << 1 | 1ULL);  // at most j mismatches
            prev = R[j];
            R[j] = newR;
        }
        if ((R[k] >> (m - 1)) & 1) occ.push_back(t - m + 1);
    }
    return occ;
}
```

12.2 Edit-distance k-differences (Landau–Vishkin)

For edit distance (substitutions, insertions, deletions), the diagonal-based algorithm of Landau and Vishkin (1989) achieves $\mathcal{O}(|T| \cdot k)$. The idea: store, for each diagonal d and each error budget $e \in 0..k$, the furthest position in T reachable from P using at most e edits along that diagonal. The transitions are: match as far as possible (LCP), then consider each of (substitution, insertion, deletion) as a transition to the next error level.

Landau-Vishkin k-difference (skeletal)

```cpp
// For each diagonal d = i - j (text position - pattern position),
// L[e][d] = the largest i such that we can match P[0..j] with T[0..i]
// using at most e edits. We use LCP queries to skip equal prefixes.
int landauVishkin(const std::string& T, const std::string& P, int k) {
    int n = T.size(), m = P.size();
    // L[e] indexed by diagonal d in [-m, n]
    std::vector<std::unordered_map<int,int>> L(k + 1);
    L[0][0] = 0;
    // ... (implementation requires LCP structure on T#P, see textbooks)
    // Return the minimum e in 0..k such that L[e][n - m] reaches n.
    return -1;  // not implemented in this skeletal sketch
}
```

12.3 Myers' bit-vector k-difference

Combine Landau–Vishkin's diagonal structure with Myers' bit-vector DP: maintain a bit-vector per diagonal, processing all diagonals simultaneously. The result is $\mathcal{O}(|T| \cdot k/w)$ for $k \leq w$, the fastest known algorithm in practice.

12.4 When to use approximate matching

When to use

- **Hamming, $|P| \le 64$, small k**: bit-parallel k-mismatch is the shortest code.
- **Hamming, $|P| > 64$, small k**: Landau–Vishkin gives $\mathcal{O}(|T| \cdot k)$.
- **Edit distance, $|P| \le 64$**: Myers' bit-vector $\mathcal{O}(|T|)$.
- **Edit distance, $|P| > 64$, small k**: Landau–Vishkin $\mathcal{O}(|T| \cdot k)$ with LCP via suffix array.
- **Edit distance, no k bound**: $\mathcal{O}(|T| \cdot |P|)$ standard DP, or $\mathcal{O}(|T| \cdot |P|/w)$ with bit-parallel block-Myers.

12.5 Codeforces references

- **CF 5E — Subblock**: DP on the longest increasing subsequence, bit-parallel.
- **CF 1197D**: distance-bounded substring matching.
- **CF 514C**: 1-error matching with trie + ad-hoc (can also be done with bit-parallel).

13 Palindromic Factorisation via Eertree Series Links

The minimum palindromic factorisation of S is the smallest k such that $S = w_1 w_2 \dots w_k$ where each w_i is a palindrome. The naive DP $\mathrm{dp}[i] = \min_{j:S[j..i] \text{ is palindrome}} \mathrm{dp}[j-1] + 1$ is $\mathcal{O}(n^2)$. The expert version using the eertree's *series links* achieves $\mathcal{O}(n \log n)$, and with careful engineering $\mathcal{O}(n)$.

13.1 Series links: the key data structure

Definition 13.1 (Series link)**.** For each eertree state v (representing a palindromic substring of length $\mathrm{len}(v)$), its *series link* $\mathrm{slink}(v)$ is the longest proper palindromic suffix of v whose δ-value differs from v's. Here $\delta(v) = \mathrm{len}(v) - \mathrm{len}(\mathrm{link}(v))$ is the "delta gap".

The series links partition the palindromic suffixes of every prefix into $\mathcal{O}(\log n)$ groups (because δ-values double along the series chain). Each group consists of palindromes whose δ-values are all equal — these form an arithmetic progression of lengths.

13.2 The $\mathcal{O}(n \log n)$ algorithm

Theorem 13.2 (Eppstein 1992, palindromic factorisation)**.** *The minimum palindromic factorisation of S can be computed in $\mathcal{O}(n \log n)$ time using eertree + series links.*

Proof sketch

Maintain, alongside the eertree, two values per state: $g[v]$ = the optimal factorisation length using palindromic suffixes in the chain ending at v, and $h[v]$ = the contribution of the series-link group. Update each by walking the series-link chain (length $\le \log n$). The DP transition is

$$\mathrm{dp}[i] = \min_{v \text{ palindromic suffix of } S[0..i]} (\mathrm{dp}[i - \mathrm{len}(v)] + 1),$$

and the series links let us express this minimum as a min over $\mathcal{O}(\log n)$ arithmetic progressions per position i, computed in $\mathcal{O}(1)$ each.

Palindromic factorisation via series links (skeletal)

```cpp
struct EertreeFactorisation {
    // Standard eertree (see primer) extended with series links.
    struct Node {
        int len, link, slink;
        int delta;              // len - len[link]
        int g;                  // group DP value
        std::map<char, int> next;
    };
    std::vector<Node> t;
    std::string s;
    int last;
    std::vector<int> dp;        // dp[i] = min pal factorisation of S[0..i]

    EertreeFactorisation() {
        t.push_back({0, 1, 1, 0, 0, {}});  // empty
        t.push_back({-1, 1, 1, 0, 0, {}}); // imaginary
        last = 0;
    }
    int getLink(int v, int i) {
        while (i - t[v].len - 1 < 0 || s[i - t[v].len - 1] != s[i])
            v = t[v].link;
        return v;
    }
    void extend(char c) {
        s.push_back(c);
        int i = (int)s.size() - 1;
        int cur = getLink(last, i);
        if (!t[cur].next.count(c)) {
            int now = (int)t.size();
            Node node;
            node.len = t[cur].len + 2;
            // set up link (suffix link) as in primer...
            // set up slink (series link)
            if (t[now].link != 0 && t[now].link != 1) {
                int q = t[t[now].link];
                if (q.delta == t[now].delta - q.len) // careful: equality test on delta
                    node.slink = q.slink;
                else
                    node.slink = t[now].link;
            } else {
                node.slink = 0;
            }
            t.push_back(node);
        }
        // DP update: walk series-link chain; for each, take min over the chain
        int v = last;
        while (v > 1) {
            // ... (handle the chain, O(log n) per character)
            v = t[v].slink;
        }
    }
};
```

13.3 When to use the palindromic factorisation

When to use

Use the series-link algorithm when the problem explicitly asks for the *minimum* number of palindromic factors. Other variants:

- **Even-length-only palindromic factorisation** (CF 932G): rephrase S as $s_1 s_n s_2 s_{n-1} \dots$ so that every even-length factor of the new string corresponds to a palindromic factor of S; then run the standard palindromic factorisation.
- **Count palindromic factorisations** (modular arithmetic): variant of the same DP, replacing min with +.
- **Bounded palindromic factors** ($w_i \le L$): add a constraint to the series-link walk; same complexity.

13.4 Codeforces references

- **CF 932G — Palindrome Partition**: the canonical eertree + series-links problem.
- **CF 17E — Palisection**: count pairs of overlapping palindromic substrings; eertree + DP.

14 Trie Tricks: XOR, Persistent, Merge, and Heavy-Light on Trie

The primer introduced XOR tries, persistent trie, and small-to-large merge. The expert additions are heavy-light decomposition on the trie (for "k-th lexicographically smallest path" queries) and the trie + bitmask trick (for trie with character classes).

14.1 Heavy-light on trie

If the trie is large and the problem asks "for each prefix of T, the k-th lexicographically smallest extension" (with some constraint), heavy-light decomposition on the trie lets us avoid recursing on every light child. Identify the "heavy" child of each node (the one with the largest subtree); the path of heavy children forms a heavy path. Each trie node belongs to exactly one heavy path. A query descends from the root, at each step either going down the heavy path or jumping across (with a cost of $\mathcal{O}(1)$ for the heavy path and $\mathcal{O}(\log n)$ for the light jump). The result: $\mathcal{O}(\log^2 n)$ per query, vs $\mathcal{O}(n)$ for the naive descent.

14.2 Trie + bitmask (alphabet = small set)

For a small alphabet $\sigma \le 64$, store the children of each trie node as a 64-bit mask of *which* characters have children. Insert and search use bit operations to find the first 1 bit. The advantage: $\mathcal{O}(1)$ per character (one mask operation) regardless of alphabet density.

14.3 Persistent trie applications

Beyond the "k-th smallest in range" from the primer, the persistent trie supports:

- **Maximum XOR with x in $A[l..r]$**: persistent trie on bits; for each range query, walk two versions in parallel picking the bit that maximises XOR.
- **Count of values $\le x$ in $A[l..r]$**: persistent trie with subtree counts; descend following x.
- **k-th smallest across multiple subarrays**: persistent trie + standard order-statistics.

14.4 When to use expert trie tricks

When to use

- **HLD on trie**: when the trie is large and you need $\mathcal{O}(\log^2 n)$ per query.
- **Bitmask trie**: for small alphabets, the fastest trie implementation in practice.
- **Persistent trie on bits**: maximum XOR with a value in a range.
- **Trie + DP on masks**: when each trie node carries a submask-state.

15 Lyndon Words, Duval, Booth, and the Runs Theorem

The primer covered Duval factorisation and Booth minimal rotation. The expert addition is the *runs theorem* — the structural result that every string has $\mathcal{O}(n)$ runs total, enabling $\mathcal{O}(n)$ algorithms for problems that previously required $\mathcal{O}(n \log n)$ or more.

15.1 Lyndon factorisation, with the full proof sketch

Theorem 15.1 (Chen–Fox–Lyndon factorisation)**.** *Every string S admits a unique factorisation $S = w_1 w_2 \dots w_k$ where each w_i is a Lyndon word and $w_1 \geq w_2 \geq \cdots \geq w_k$. Duval's algorithm computes this in $\mathcal{O}(n)$ time.*

Proof sketch

A Lyndon word is, equivalently, a string strictly smaller than every proper rotation of itself. Lyndon words are exactly the *aperiodic necklaces* — the lexicographically smallest representatives of the rotation classes. Their factorisation is the natural "decreasing" order of these representatives; uniqueness follows from the lexicographic-ordering properties of Lyndon words.

15.2 Booth's algorithm: full statement

Theorem 15.2 (Booth 1980)**.** *The lexicographically smallest cyclic rotation of S can be found in $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ memory.*

The algorithm: maintain a candidate rotation index k (initially 0). For each $j \in 1..2n-1$, compare $S[(k+j) \bmod n]$ to $S[j \bmod n]$ using a modified failure function; if the new character is smaller, advance k. The result is the smallest rotation.

15.3 Runs theorem

Definition 15.3 (Run)**.** A *run* (also called a "maximal periodicity") in S is a triple (i, j, p) such that $S[i..j]$ has minimal period p with $j - i + 1 \geq 2p$, and $S[i..j]$ is maximal: $S[i-1..j]$ and $S[i..j+1]$ do not have period p.

Theorem 15.4 (Kolpakov–Kucherov 1999)**.** *A string of length n has at most n runs. The sum of $(j-i+1)/p$ (the "exponent") over all runs is $\mathcal{O}(n)$.*

The implication: any algorithm that processes all runs of S can run in $\mathcal{O}(n)$ total time, since the number of runs is linear and the total work per run is proportional to its exponent.

15.4 Algorithms based on runs

- **All squares in S**: a square is a substring of the form ww for non-empty w. Every square is contained in some run; the runs theorem gives $\mathcal{O}(n)$ squares (counted with multiplicity) and $\mathcal{O}(n)$ time to enumerate them.
- **All maximal repetitions**: same.
- **Local periods**: every position has at most $\mathcal{O}(\log n)$ local periods; runs enumerate them.

15.5 When to use Lyndon / Booth / runs

When to use

- **Smallest cyclic rotation**: Booth (preferred for $\mathcal{O}(n)$ worst case) or Duval (preferred for shortness).
- **Lexicographically smallest suffix**: Duval (last factor).
- **Count distinct rotations**: Duval + Lyndon factorisation gives the answer.
- **All squares in S**: runs enumeration (rare in CP, but IOI-level).
- **String compression via Lyndon words**: when the problem asks for the Lyndon structure explicitly.

15.6 Codeforces references

- **CF 1026D**: cyclic rotation matching, Booth or Duval.
- **Emax blog on Lyndon words**: canonical reference.
- **CF 452F — Permutation**: hashing + Lyndon-style inversion check.

16 Suffix Array Applications: A Worked-Problem Compendium

This chapter walks through five classic problems that all reduce to suffix-array + LCP machinery. For each, we give the problem statement, the reduction, the algorithm, the complexity, and a Codeforces reference.

16.1 Problem 1: Longest repeated substring

Statement. Given S, find the longest string that occurs at least twice in S as a substring.

Reduction. A repeated substring is a common prefix of two distinct suffixes of S. The longest such substring is the maximum entry in the LCP array.

Algorithm. Build the suffix array sa and LCP array lcp of S (with the primer's $\mathcal{O}(n \log^2 n)$ construction and Kasai's $\mathcal{O}(n)$ LCP). Scan lcp for its maximum value m; the answer is $S[\text{sa}[i^*] \,..\, \text{sa}[i^*]+m-1]$ where i^* is the index achieving the maximum.

Complexity. $\mathcal{O}(n \log n)$ for the SA build, $\mathcal{O}(n)$ for the scan; total $\mathcal{O}(n \log n)$.

Worked example. $S =$ `abacaba`, sa $= [6, 4, 0, 2, 5, 1, 3]$, lcp $= [1, 3, 1, 0, 2, 0]$. The maximum is 3 at index 1, corresponding to sa$[1] = 4$: the substring $S[4..6] =$ `aba` is the longest repeated substring. (Indeed `aba` occurs at positions 0 and 4 in S.)

16.2 Problem 2: k-th lexicographically smallest substring

Statement. Given S and k, find the k-th smallest substring of S in lexicographic order. Distinct substrings only.

Reduction. The distinct substrings are in one-to-one correspondence with the (suffix, length) pairs that are not prefixes of any longer LCP-matching substring. We can iterate them in sorted order by walking the SA: starting from the shortest prefix of sa[0], then prefixes of sa[0] in increasing length until we hit lcp[1], then prefixes of sa[1] from length lcp[1] + 1 onwards, and so on.

Algorithm.

```
long long countUpTo = 0;
for (int i = 0; i < n; ++i) {
    int suffixStart = sa[i];
    int suffixLen   = n - suffixStart;
    int startLen    = (i == 0) ? 1 : lcp[i - 1] + 1;
    // prefixes of length startLen .. suffixLen are new distinct substrings
    long long block = suffixLen - startLen + 1;
    if (countUpTo + block >= k) {
        int targetLen = startLen + (int)(k - countUpTo - 1);
        return S.substr(suffixStart, targetLen);
    }
    countUpTo += block;
}
```

Complexity. $\mathcal{O}(n)$ after the $\mathcal{O}(n \log n)$ SA build. For finding k directly with binary search, $\mathcal{O}(n \log n)$.

16.3 Problem 3: Number of distinct substrings

Statement. Count the number of distinct substrings of S.

Reduction. From the primer: $\binom{n+1}{2} - \sum_{i=1}^{n-1} \text{lcp}[i]$.

Algorithm. Build SA + LCP, evaluate the formula.

Worked example. $S =$ `abacaba`, $n = 7$, $\sum \text{lcp} = 7$. Answer: $\binom{8}{2} - 7 = 28 - 7 = 21$.

16.4 Problem 4: Longest common substring of two strings

Statement. Given S and T, find the longest string that is a substring of both.

Reduction. Concatenate $X = S \# T$ with a separator $\#$ smaller than every character. Build SA + LCP. For each pair of adjacent suffixes in the SA, if one starts in S and the other in T, their LCP is a candidate common substring. The maximum such LCP is the answer.

Algorithm. Walk the SA; for each $i \geq 1$, let $a = \text{sa}[i-1]$, $b = \text{sa}[i]$. If exactly one of a, b is $\leq |S|$ (i.e. in the S part), then $\text{lcp}[i-1]$ is a candidate; track the max.

Worked example. $S =$ `abacab`, $T =$ `caba`. Concatenate $X =$ `abacab#caba`. The SA's adjacent pairs include suffixes from both halves; the max cross-half LCP is 3 (the substring `cab`, shared by both strings).

Complexity. $\mathcal{O}((|S| + |T|) \log(|S| + |T|))$.

16.5 Problem 5: Substring frequency queries

Statement. Preprocess S to answer, for each query pattern P, the number of occurrences of P in S.

Reduction. The occurrences of P in S form a contiguous interval in the suffix array. Binary search to find the smallest i such that $S[\text{sa}[i]..]$ is $\geq P$, and the smallest j such that $S[\text{sa}[j]..]$ is $> P$ in the first-$(|P|)$-characters sense. The count is $j - i$.

Algorithm. Use the LCP-augmented binary search: at each step, compare P with the current SA suffix using the cached LCP. The total time is $\mathcal{O}(|P| + \log n)$ per query.

Complexity. $\mathcal{O}(n \log n)$ preprocessing, $\mathcal{O}(|P| + \log n)$ per query. With an LCP sparse table, $\mathcal{O}(|P| \cdot \log n)$ or amortised $\mathcal{O}(|P| + \log n)$.

16.6 Codeforces references

- **CF 1238D**: SA + LCP for substring queries.
- **CF 427D**: classic LCS via SA.
- **CF 873E**: SA + bitmask for substring grade queries.
- **CF 1066F**: SA + LCP + sparse table for substring comparisons.

17 Suffix Automaton Applications: A Worked-Problem Compendium

Where the previous chapter showcased SA + LCP, this chapter does the same for the suffix automaton. The recurring theme: SAM gives online answers where SA requires precomputation, and SAM gives shorter code where SA needs LCP-sparse-tables.

17.1 Problem 1: Online substring existence

Statement. Process S character-by-character; after each character, answer queries "is T a substring of the current S?".

Algorithm. Maintain SAM as S grows. For each query, walk the SAM from the root following characters of T; if the walk succeeds, T is a substring.

Complexity. $\mathcal{O}(1)$ amortised per character; $\mathcal{O}(|T|)$ per query.

17.2 Problem 2: Number of distinct substrings ending at each prefix

Statement. For each $i \in 0..n-1$, count the number of distinct substrings of $S[0..i]$.

Algorithm. After each extend(c), the number of new distinct substrings is len(cur)−len(link(cur)). The cumulative answer for prefix i is the sum of these up to step i.

Complexity. $\mathcal{O}(n)$ total.

17.3 Problem 3: Number of occurrences of each substring

Statement. For each substring T of S, count how many times T occurs in S.

Algorithm. Build SAM. For each state v, set $\text{cnt}(v) = 1$ if v is not a clone (i.e. it was created by an extend call), else $\text{cnt}(v) = 0$. Propagate cnt up the suffix-link tree (process states in decreasing order of len). Then for any substring T, walk the SAM to the state v containing T; the answer is $\text{cnt}(v)$.

Complexity. $\mathcal{O}(n)$ preprocessing, $\mathcal{O}(|T|)$ per query.

17.4 Problem 4: Smallest cyclic rotation in $\mathcal{O}(n)$ without Booth

Statement. Find the lexicographically smallest cyclic rotation of S.

Algorithm. Build SAM on S. From the root, repeatedly pick the alphabetically smallest available transition; do this n times. The path spells the smallest cyclic rotation.

Why it works. The SAM of S accepts all suffixes of S; concatenating with itself (in effect) gives all cyclic rotations. The greedy walk picks the smallest at each step.

Complexity. $\mathcal{O}(n)$ total. Slightly slower in practice than Booth (constant factor) but simpler to remember if you already have SAM in your template.

17.5 Problem 5: Distinct substrings of S that occur in T

Statement. Given S and T, count the number of distinct substrings of S that also occur as substrings of T.

Algorithm. Build SAM on T. Walk through S in the SAM, tracking the current state and matched length; whenever we are at state v with matched length l, all substrings $S[i..i+l'-1]$ for $l' \le l$ are substrings of T. Use a hash set over (state, length) pairs to count distinct ones.

Complexity. $\mathcal{O}(|T|+|S|)$ with a hash set, $\mathcal{O}(|T|+|S|\log|S|)$ with a balanced tree.

17.6 Codeforces references

- **CF 235C**: cyclic-shift occurrence counting.
- **CF 204E**: SAM + suffix-link tree DP for multi-string substring counting.
- **CF 432D**: count occurrences of each prefix-suffix.

18 Combinatorics on Words: Necklaces, Primitive Words, and de Bruijn Sequences

Combinatorics on words is the algebraic side of string algorithms. Three classes of objects appear regularly in CP problems: *Lyndon words*, *primitive words*, and *de Bruijn sequences*. We introduce them and give CP-style algorithms for each.

18.1 Necklaces and Lyndon words

Definition 18.1 (Necklace)**.** A *necklace* is an equivalence class of strings under cyclic rotation. The *Lyndon representative* of a necklace is the lexicographically smallest rotation; this is always a Lyndon word (or a single character).

The number of necklaces of length n over an alphabet of size σ is given by the Burnside-count formula

$$M(n,\sigma) \;=\; \frac{1}{n}\sum_{d\mid n}\varphi(d)\sigma^{n/d}.$$

For $n=4$ and $\sigma=2$, this gives $M(4,2)=\frac{1}{4}(\varphi(1)\cdot 2^4+\varphi(2)\cdot 2^2+\varphi(4)\cdot 2^1)=\frac{1}{4}(16+4+4)=6$.

18.2 Primitive words

Definition 18.2 (Primitive word)**.** A string S is *primitive* if it is not a power of a shorter string: $S \neq T^k$ for any $k \ge 2$.

Proposition 18.3. *A string S is primitive iff* $\gcd(n, \pi[n-1]) = 1$ *where π is the prefix function.*

Proof sketch

If $\pi[n-1] = n-p$ with p the smallest period, then S has period p. By the periodicity lemma, S has period $\gcd(n,p)$. S is primitive iff $p=n$, iff $\gcd(n,p)=1$.

18.3 de Bruijn sequences

Definition 18.4 (de Bruijn sequence)**.** A *de Bruijn sequence* of order k over an alphabet Σ is a cyclic string $B(\Sigma,k)$ in which every possible length-k string over Σ appears exactly once as a substring. Its length is $|\Sigma|^k$.

For $\Sigma=\{0,1\}$ and $k=3$, $B(\Sigma,3)=$ `00010111` (cyclically), and every length-3 binary string occurs exactly once: `000`, `001`, `010`, `101`, `011`, `111`, `110`, `100`.

de Bruijn sequence via Hierholzer on de Bruijn graph

```cpp
#include <set>
// Build a de Bruijn sequence B(Sigma, k) by Eulerian-cycle DFS on the
// de Bruijn graph G(Sigma, k): nodes = strings of length k-1,
// edges = strings of length k, edge (u0..u_{k-2}) -> (u1..u_{k-1})
// labelled by the appended character.
std::string deBruijn(int k, int sigma) {
    std::string seq;
    std::set<std::string> visited;
    // recursive DFS; iterative is also fine
    std::function<void(std::string)> dfs = [&](std::string u) {
        for (int c = 0; c < sigma; ++c) {
            std::string v = u.substr(1) + std::string(1, 'a' + c);
            if (!visited.count(u + (char)('a' + c))) {
                visited.insert(u + (char)('a' + c));
                dfs(v);
            }
        }
        seq.push_back(u.back());  // Hierholzer: append on the way back
    };
    dfs(std::string(k - 1, 'a'));
    return std::string(seq.rbegin(), seq.rend());  // reverse
}
```

18.4 When to use combinatorics on words

When to use

- **Count necklaces** of given length and alphabet size (often a subproblem in combinatorial counting problems).
- **Find the Lyndon representative of a string** (Booth or Duval).
- **Test if S is primitive** (prefix function check).
- **Generate a de Bruijn sequence** for cracking combination locks, generating test data, or building covering sequences in CP.

18.5 Codeforces references

- **CF 81A — Plug-in**: not directly de Bruijn, but uses graph Euler-cycle.
- **CF 42C — String Tale**: necklaces and cyclic strings.

19 Streaming String Algorithms

Streaming algorithms process the input in a single left-to-right pass, with sublinear (typically $\mathcal{O}(\log n)$ or $\mathcal{O}(\sqrt{n})$) memory. For string problems, this is the territory of KMP failure automaton + bitvector sketches, Rabin–Karp with randomised bases, and the Flajolet–Martin sketch for distinct-element estimation.

19.1 Pattern matching in a stream

For a single pattern P and a streaming text T, the failure automaton from Chapter 3 is the right tool: $\mathcal{O}(|P| \cdot |\Sigma|)$ preprocessing, $\mathcal{O}(1)$ per character, $\mathcal{O}(|P| \cdot |\Sigma|)$ memory. For multiple patterns, Aho–Corasick generalises this with $\mathcal{O}(\sum |P_i| \cdot |\Sigma|)$ memory.

19.2 Distinct substrings in a stream (estimation)

If we cannot store the full SAM (it is $\mathcal{O}(n)$ memory), we use a *sketch*: maintain a k-length rolling hash for each suffix of T seen so far, and estimate the number of distinct substrings using Flajolet–Martin. The estimate is within a factor of $1+\varepsilon$ for $k = \mathcal{O}(\log n/\varepsilon^2)$ hash functions.

19.3 Approximate membership in a stream

Bloom filters are the canonical streaming string data structure: $\mathcal{O}(k)$ bits and k hash functions per element. For "is T a substring of S?" where S is streaming, use a Bloom filter on the rolling hashes of all substrings of S up to length L. False positives are controlled by k and the filter size; false negatives are impossible.

19.4 When to use streaming string algorithms

When to use

Use streaming algorithms when:

- The text is too large to fit in memory ($n \geq 10^9$).
- The problem explicitly asks for single-pass algorithms (CP problems rarely do, but research-style problems do).
- The problem is about online substring matching with limited memory (failure automaton + small cache).

Avoid streaming for typical CP problems where $\mathcal{O}(n)$ memory is acceptable.

20 Suffix Tree + LCA: $\mathcal{O}(1)$ LCP Queries

The suffix tree provides $\mathcal{O}(1)$ LCP queries via Euler-tour + sparse table on the LCA (lowest common ancestor) of two leaves. This is the historical ancestor of the LCP + sparse-table approach for the suffix array.

20.1 Euler tour

For a tree T with n nodes, the Euler tour is a sequence of length $2n-1$ obtained by DFS: each time we visit a node (including the start and re-visits), append it. The LCA of nodes u and v is the node with minimum depth in the Euler tour between the first visit of u (or v) and the first visit of v (or u).

20.2 Reduction to RMQ

Build the Euler tour array E (length $2n-1$) and the depth array D (length $2n-1$) where $D[i]$ is the depth of $E[i]$. For each node u, also store $\text{first}[u]$ = the smallest index i with $E[i] = u$. Then

$$\text{LCA}(u,v) \;=\; E\big[\text{RMQ}_D(\min(\text{first}[u],\text{first}[v]),\max(\text{first}[u],\text{first}[v]))\big].$$

The RMQ is solved by a sparse table over D.

20.3 LCP from LCA

For the suffix tree of S, the leaves correspond to the suffixes. The LCA of the leaves for $S[i..]$ and $S[j..]$ gives the LCP of these two suffixes, since the LCA node's depth is exactly the LCP length.

20.4 Complexity

- Suffix tree build: $\mathcal{O}(n)$ (Ukkonen).
- Euler tour + first[] arrays: $\mathcal{O}(n)$.
- Sparse table on D: $\mathcal{O}(n \log n)$ time and memory.
- Per LCP query: $\mathcal{O}(1)$.

20.5 When to use the suffix tree + LCA approach

When to use

- When you need $\mathcal{O}(1)$ LCP queries on the same S but cannot afford the $\mathcal{O}(n \log n)$ memory of the SA sparse table (suffix tree is $\mathcal{O}(n)$).
- When the problem explicitly reduces to LCA on suffixes (e.g. "count the number of distinct substrings that are common to S_1 and S_2 at positions i and j").
- For theoretical elegance, e.g. when writing a paper.

In CP, the SA + LCP + sparse table is more common because it's shorter to code and the memory difference is minor. The suffix-tree approach is for problems that also need other suffix-tree applications (occurrence enumeration, substring DP).

21 Wavelet Trees and Inverted Indexes

The wavelet tree is the workhorse of compressed full-text indexing. It encodes a sequence over an alphabet Σ in $n \log \sigma$ bits while supporting access, rank_c, and select_c operations in $\mathcal{O}(\log \sigma)$ time each. It is the substrate of the FM-index's Occ table.

21.1 Definition

Definition 21.1 (Wavelet tree)**.** The *wavelet tree* of a string S over Σ is a binary tree where each node holds a bit-vector. The root represents all of S over Σ; each node partitions its alphabet into two halves based on a bit, stores the resulting bit-vector (1 if char goes right, 0 if left), and recurses on each half. Leaves correspond to individual characters.

21.2 Operations

- access(i): character $S[i]$. Walk from the root: at each node, check the i-th bit; if 0, descend left with index $\text{rank}_0(i)$; else descend right with $\text{rank}_1(i)$. The leaf gives the character.
- $\text{rank}_c(i)$: number of occurrences of c in $S[0..i-1]$. Walk from the root: at each node, check whether c goes left or right; if left, $i \leftarrow \text{rank}_0(i)$; else $i \leftarrow \text{rank}_1(i)$. The result is the rank at the leaf.
- $\text{select}_c(k)$: position of the k-th occurrence of c. Walk from the leaf corresponding to c up to the root, undoing the rank operations.

21.3 Bit-vector rank/select in $\mathcal{O}(1)$

Each bit-vector in the wavelet tree is augmented with a rank/select index that supports $\mathcal{O}(1)$ queries after $\mathcal{O}(n)$ preprocessing. The standard implementation is the *RRR* structure (Raman–Raman–Rao 2007), which uses $nH_0 + o(n)$ bits where H_0 is the 0-order empirical entropy.

21.4 CP application: Occ table for FM-index

The wavelet tree on the BWT supports $\mathrm{Occ}(c, i) = \mathrm{rank}_c(i)$ in $\mathcal{O}(\log \sigma)$, replacing the naive $n\sigma$ integer table. The FM-index then becomes a true $\mathcal{O}(n \log \sigma)$ bit full-text index.

21.5 When to use wavelet trees

When to use

Use the wavelet tree when:

- You need rank/select on a string and memory is tight.
- You are building the FM-index's Occ table.
- You need *range queries on a string* (e.g. "count occurrences of c in $S[l..r]$") — this is $\mathrm{rank}_c(r+1) - \mathrm{rank}_c(l)$ in $\mathcal{O}(\log \sigma)$.

22 String Dynamic Programming: A Compendium

A huge class of CP string problems reduce to dynamic programming on one or two indices of the string. This chapter collects the major DP families: edit distance and its variants, regular-expression matching, subsequence problems, and string partitioning.

22.1 Levenshtein edit distance, with backtracking

The classical edit distance $\mathrm{ed}(A, B)$ is the minimum number of insertions, deletions, and substitutions to transform A into B. The DP is

$$\mathrm{dp}[i][j] = \min \begin{cases} \mathrm{dp}[i-1][j] + 1 & \text{(delete } A[i]\text{)}, \\ \mathrm{dp}[i][j-1] + 1 & \text{(insert } B[j]\text{)}, \\ \mathrm{dp}[i-1][j-1] + [A[i] \neq B[j]] & \text{(match/substitute)}. \end{cases}$$

Base case: $\mathrm{dp}[0][j] = j$, $\mathrm{dp}[i][0] = i$. Total $\mathcal{O}(|A| \cdot |B|)$. To backtrack the edit operations, store which case achieved the minimum.

Levenshtein edit distance with full table

```cpp
int editDistance(const std::string& A, const std::string& B) {
    int n = A.size(), m = B.size();
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(m + 1));
    for (int i = 0; i <= n; ++i) dp[i][0] = i;
    for (int j = 0; j <= m; ++j) dp[0][j] = j;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j) {
            int cost = (A[i-1] == B[j-1]) ? 0 : 1;
            dp[i][j] = std::min({dp[i-1][j] + 1, dp[i][j-1] + 1,
                                 dp[i-1][j-1] + cost});
        }
    return dp[n][m];
}
```

22.2 Damerau–Levenshtein (with transpositions)

Adds a fourth operation: transposition of two adjacent characters (so `ab` → `ba` counts as one edit, not two). The DP gets one extra case: if $i \geq 2$ and $j \geq 2$ and $A[i-2..i-1] = B[j-2..j-1]$ reversed, take $\mathrm{dp}[i-2][j-2] + 1$.

22.3 Weighted edit distance

If the cost of each substitution depends on the characters involved (e.g. `a` → `b` is cheaper than `a` → `z`), use a cost matrix $C[c_1][c_2]$ in the DP. The DP transition becomes $\text{dp}[i-1][j-1]+C[A[i]][B[j]]$ for substitution. Useful for OCR and bioinformatics (some base substitutions are more likely than others).

22.4 Longest common subsequence (LCS)

$$\text{lcs}[i][j] = \max \begin{cases} \text{lcs}[i-1][j] & \text{skip } A[i], \\ \text{lcs}[i][j-1] & \text{skip } B[j], \\ \text{lcs}[i-1][j-1] + [A[i] = B[j]] & \text{match.} \end{cases}$$

For $|A| = |B| = n$, the standard $\mathcal{O}(n^2)$ DP is acceptable up to $n \approx 5000$. For $n \le 5 \cdot 10^4$, use the bit-parallel LCS that achieves $\mathcal{O}(n^2/w)$ via a bitset representation of B's character positions.

22.5 Shortest common supersequence

$\text{scs}(A, B) = |A| + |B| - \text{lcs}(A, B)$. So compute LCS first, then subtract.

22.6 Number of distinct LCS

To count distinct LCSs (not just the length), add a second DP: $\text{cnt}[i][j]$ = number of distinct LCSs of $A[0..i]$ and $B[0..j]$ that have length $\text{lcs}[i][j]$. The transition is a careful union-find over the match positions; the result is computed modulo p for large alphabets.

22.7 Regular expression matching

Given a regex R (with $\cdot$, $|$, $*$) and a text T, does R match some substring of T? The Thompson construction converts R to an NFA in $\mathcal{O}(|R|)$; the NFA simulation runs in $\mathcal{O}(|T| \cdot |R|)$ per match attempt, or $\mathcal{O}(|T| \cdot |R|)$ for all matches. With bit-parallel NFA simulation (the Shift-Or trick applied to the NFA), this drops to $\mathcal{O}(|T| \cdot |R|/w)$.

22.8 String partitioning DP

"Partition S into contiguous pieces such that ..." (e.g. "minimise the sum of palindromic-factor costs" or "maximise the count of pieces equal to a dictionary word"). The DP is $\text{dp}[i] = \min / \max_{j<i} \text{dp}[j] + \text{cost}(S[j+1..i])$. The complexity is $\mathcal{O}(n^2)$ unless the cost function admits a faster structure (e.g. SAM or AC for dictionary lookups, hashing for substring tests).

22.9 DP on cyclic strings

For problems on cyclic rotations of S, the trick is to compute the DP on $S + S$ and then take the minimum / maximum over all n windows of length n. Beware of double-counting: $S + S$ has $2n$ positions, but cyclic shifts are only n distinct strings; pick a canonical representative.

22.10 When to use string DP

When to use

- **Edit distance** (Levenshtein, Damerau–Levenshtein, weighted) for bioinformatics and OCR.
- **LCS / SCS** for diff tools and merge algorithms.
- **Regex matching** for grep-like problems.

- **String partitioning** with a custom cost function (e.g. palindromic partition).
- **DP on cyclic strings** when the problem is rotation-invariant.

For $n \le 5000$ use the naive $\mathcal{O}(n^2)$ DP. For $n \le 5 \cdot 10^4$ use bit-parallel. For $n \ge 10^5$, look for a structural reduction to a suffix-automaton or suffix-tree algorithm.

22.11 Codeforces references

- **CF 1132F — Clear the String**: classic interval DP on strings.
- **CF 1114D — Flood Fill**: similar interval DP.
- **CF 5E — Subblock**: bit-parallel LIS.
- **CF 1197D**: distance-bounded substring matching.

23 Hashing: Advanced Applications

This chapter collects five worked examples of polynomial hashing beyond the basic substring equality test. Each represents a recurring CP problem family that is much harder to solve with suffix arrays or SAM.

23.1 Application 1: Longest palindromic substring via hashing + binary search

Statement. Find the longest palindromic substring of S.

Algorithm. Binary search on length L. For each L, use bidirectional hashing to check if any length-L substring is a palindrome: for each starting position i, compare $H_f(S[i..i+L-1])$ with $H_r(S[i..i+L-1])$ (with appropriate scaling). The palindrome test is $\mathcal{O}(1)$ per starting position, $\mathcal{O}(n)$ per length, $\mathcal{O}(n \log n)$ total.

Why not Manacher? Manacher is $\mathcal{O}(n)$ and shorter. The hashing version is competitive when you already have hashing in your template but not Manacher. As an exercise, this is a useful illustration of how the bidirectional hash structure works.

23.2 Application 2: Number of distinct substrings of length exactly k

Statement. Count the number of distinct substrings of S of length k.

Algorithm. Hash every length-k substring, insert into a hash set, return the set's size. $\mathcal{O}(n)$ time per query, $\mathcal{O}(n)$ memory.

23.3 Application 3: Longest common substring of k strings

Statement. Find the longest string that is a substring of all k given strings.

Algorithm. Binary search on length L. For each L, hash all length-L substrings of S_1 into a hash set; for $S_2, \ldots, S_k$, intersect with the previous set. If non-empty after k intersections, L is achievable. $\mathcal{O}(k \cdot n \log n)$ total.

23.4 Application 4: Number of positions where S and T differ, with at most one wildcard

Statement. Given S and T of equal length, count the positions where $S[i] \neq T[i]$, with the option to ignore one position as a wildcard.

Algorithm. Use hashing to binary-search for the first mismatch position in $\mathcal{O}(\log n)$, then skip that position and binary-search the next mismatch. $\mathcal{O}(\log n)$ per mismatch; total $\mathcal{O}(k \log n)$ for k mismatches.

23.5 Application 5: Substring equality queries on a dynamic string (with rollback)

Statement. S is subject to point updates (replace $S[i]$ with a new character); after each update, answer substring-equality queries.

Algorithm. Use a treap-based rolling hash where each node stores the hash of its subtree. Point updates are $\mathcal{O}(\log n)$; substring hash queries are $\mathcal{O}(\log n)$. This is the canonical "dynamic string" structure for CP.

23.6 When to use hashing, advanced

When to use

The advanced hashing applications are most useful when:

- The problem is naturally expressed as "many substring equality queries" but the alphabet size makes SA construction infeasible.
- The string is dynamic (point updates) and SA cannot be re-built efficiently.
- The problem requires bidirectional hash (palindrome tests, mirror-image queries).

24 Aho–Corasick DP Variants

The Aho–Corasick automaton is the substrate for many CP problems that look like "walk through T subject to some constraint involving the patterns $P_1, \ldots, P_k$". This chapter collects four DP families beyond the basic AC + DP for "avoid banned words".

24.1 Variant 1: Count strings of length n containing each P_i at least once

Statement. Count the number of strings X of length n over Σ such that each P_i occurs in X at least once.

Algorithm. Use inclusion–exclusion on the set of patterns NOT present. For each subset $S \subseteq \{1, \ldots, k\}$, compute the number of strings avoiding every P_i with $i \in S$ (this is the basic AC + matrix exponentiation). Sum with $(-1)^{|S|}$ signs. Total: $\mathcal{O}(2^k \cdot |Q|^3 \log n)$ — practical for $k \leq 12$.

24.2 Variant 2: Maximum-score walk through T where each P_i contributes a weight

Statement. Each pattern P_i has a weight w_i. A string T has a score equal to the sum of weights of patterns that occur in T. Find the maximum score over all walks of length n.

Algorithm. Build AC; for each state v, compute the total weight $W(v)$ of patterns that end at v or in v's output-link chain. DP: $\text{dp}[i][v]$ = max score of a length-i walk ending in state v. Transition: $\text{dp}[i+1][\delta(v,c)] = \max(\text{dp}[i][v] + W(\delta(v,c)))$. With matrix exponentiation: $\mathcal{O}(|Q|^3 \log n)$.

24.3 Variant 3: Shortest string containing all P_i

Statement. Find the shortest string that contains every P_i as a substring.

Algorithm. Build AC; for each pair of states u, v, compute the shortest path from u to v that visits each "pattern-end state" at least once. This is a TSP on the AC automaton with 2^k state. Use bitmask DP: $\text{dp}[\text{mask}][v]$ = shortest length to reach state v having visited the patterns in mask. Transition: $\text{dp}[\text{mask}|(1 \ll \text{pat}(v))][v] = \min(\text{dp}[\text{mask}][u] + \text{dist}(u, v))$ where dist is precomputed BFS. Total: $\mathcal{O}(|Q| \cdot 2^k + |Q|^2)$.

24.4 Variant 4: Number of occurrences of P_i in T

Statement. For each i, count the number of occurrences of P_i in T.

Algorithm. This is just Aho–Corasick matching. The output is a vector of counts, one per pattern. The complexity is $\mathcal{O}(|T| + \sum |P_i| + \text{total occurrences})$.

24.5 When to use AC DP variants

When to use

Use AC + DP when:

- The problem asks for the count of strings of length n with some pattern-avoidance / pattern-containment property.
- The problem is about shortest / longest / maximum-score walks through a text with patterns as features.
- The problem is about "travelling salesman on patterns" — shortest string containing all P_i.

The common thread: the AC automaton encodes "which patterns have been seen recently" as a single state, and the DP iterates over (length, state) pairs.

24.6 Codeforces references

- **CF 1202E**: count triples A, B with $A + B \subseteq T$ — the canonical AC + DP.
- **CF 86C**: count strings of length n containing a pattern — AC + matrix.
- **CF 1003F**: AC + shortest-superstring.

25 Practice Problems with Full Solutions

This chapter presents five representative problems, each with the full pattern-recognition reasoning and a reference solution. The aim is to train the reader's instinct: read the statement, identify the hybrid pattern, write the structure.

25.1 Problem 1: Two-string longest common substring

Statement. Given S and T ($|S|, |T| \le 10^5$), find the longest common substring.

Pattern recognition. "Substring of S that is also a substring of T" → suffix array on $S\#T$ + LCP scan; alternatively, SAM on S and walk T.

Reference solution (SAM). Build SAM on S. Walk T through the SAM tracking (current state, current length). On mismatch, follow suffix links until a transition is available or we reach the root; adjust the length accordingly. The maximum length seen is the answer. $\mathcal{O}(|S| + |T|)$.

25.2 Problem 2: Number of distinct substrings of S

Statement. Count distinct substrings of S ($|S| \le 10^5$).

Pattern recognition. "All substrings of one string" → SA + LCP or SAM.

Reference solution (SA). Build SA + LCP. Answer: $\binom{n+1}{2} - \sum_i \text{lcp}[i]$. $\mathcal{O}(n \log n)$.

Reference solution (SAM). Build SAM. Answer: $\sum_{v \neq \text{root}} (\text{len}(v) - \text{len}(\text{link}(v)))$. $\mathcal{O}(n)$.

25.3 Problem 3: Minimum palindromic factorisation

Statement. Partition S into the minimum number of palindromic substrings.

Pattern recognition. "Palindromic substrings" → eertree; "minimum factorisation" → DP with the series-link structure.

Reference solution. Build eertree with series links. DP: $\text{dp}[i] = \min_{v \text{ palindromic suffix of } S[0..i]} \text{dp}[i - \text{len}(v)] + 1$. Walk the series-link chain to compute the minimum in $\mathcal{O}(\log n)$ per position. Total: $\mathcal{O}(n \log n)$.

25.4 Problem 4: Count strings of length n avoiding banned words

Statement. Given banned words $P_1, \ldots, P_k$, count the strings of length n over Σ that contain none of them.

Pattern recognition. "Strings avoiding patterns" → AC automaton + matrix exponentiation.

Reference solution. Build AC. Mark banned states (directly or via output links). Construct adjacency matrix A excluding transitions to banned states. Answer: $\sum_v (A^n)[0][v]$ modulo p. $\mathcal{O}(|Q|^3 \log n)$.

25.5 Problem 5: Smallest cyclic rotation

Statement. Find the lexicographically smallest cyclic rotation of S ($|S| \le 10^6$).

Pattern recognition. "Cyclic rotation, smallest" → Booth's algorithm or Duval's factorisation.

Reference solution (Booth). Run Booth's algorithm on $S + S$ with the modified failure function. The result k is the rotation index. Output $S[k..] + S[..k-1]$. $\mathcal{O}(n)$.

25.6 Lessons

When to use

The drill reinforces the hybrid-pattern catalogue:

- Each problem's first sentence points to a structure ("substring" → SA/SAM; "palindromic" → eertree; "avoid patterns" → AC; "cyclic rotation" → Booth/Duval).
- The second sentence points to a hybrid ("minimum factorisation" → DP on the structure; "count" → matrix exponentiation).
- The combination uniquely identifies the algorithm.

26 Suffix Tree Applications: Worked Problems

This chapter collects five problems that are most naturally solved on the suffix tree (as opposed to the suffix array or suffix automaton). For each, we give the statement, the suffix-tree reduction, and a complexity analysis.

26.1 Problem 1: Number of leaves below each internal node

Statement. For each internal node v of the suffix tree, compute leaves(v) = the number of leaves in v's subtree.

Reduction. A leaf corresponds to a suffix. leaves(v) = number of suffixes that start with the substring spelled by v = number of occurrences of that substring.

Algorithm. Compute subtree leaf counts via DFS. Each leaf contributes 1 to its ancestors. $\mathcal{O}(n)$ total via bottom-up accumulation.

Application. "For each substring, count its occurrences" in $\mathcal{O}(n)$ total time, with one query per node.

26.2 Problem 2: Longest substring occurring at least k times

Statement. Find the longest substring of S that occurs at least k times.

Reduction. Each node v with leaves$(v) \geq k$ corresponds to a substring of length depth(v) occurring $\geq k$ times. The answer is the maximum depth(v) over such nodes.

Algorithm. DFS the suffix tree, computing leaves(v) bottom-up; track the max depth(v) among nodes with leaves$(v) \geq k$. $\mathcal{O}(n)$.

26.3 Problem 3: All distinct substrings starting with P

Statement. Given P, enumerate all distinct substrings of S that have P as a prefix.

Reduction. Walk the suffix tree from the root following P. If you arrive at a point on an edge or at a node v, all distinct substrings starting with P correspond to (path-from-v-to-leaf, length).

Algorithm. DFS from v enumerating all distinct suffixes of v's subtree. Total length is the number of distinct substrings starting with P, which can be large; report the count in $\mathcal{O}(1)$ or enumerate in $\mathcal{O}(\text{count})$.

26.4 Problem 4: k-th lexicographically smallest suffix of S

Statement. Given k, find the k-th smallest suffix of S.

Reduction. The leaves of the suffix tree in DFS order are the suffixes in sorted order — this is exactly the suffix array! So the k-th smallest suffix is sa$[k]$.

Algorithm. Build the suffix tree (or just the SA, more efficiently) and look up sa$[k]$. $\mathcal{O}(n)$ for Ukkonen build, $\mathcal{O}(1)$ per query.

26.5 Problem 5: Longest common substring of three strings

Statement. Find the longest string that is a substring of S_1, S_2, and S_3.

Reduction. Build a generalised suffix tree on $S_1\#_1S_2\#_2S_3\#_3$ with distinct sentinels. Each leaf is tagged with which string it comes from. For each internal node v whose subtree contains leaves from all three strings, depth(v) is a candidate length. Take the maximum.

Algorithm. DFS the generalised suffix tree; for each node, compute a 3-bit mask of which strings have leaves in its subtree. The deepest node with mask 111_2 is the answer. $\mathcal{O}(|S_1| + |S_2| + |S_3|)$.

26.6 When to use the suffix tree for these problems

When to use

The suffix tree's natural advantage over SAM is the explicit representation of *subtree-leaf structure*. For all five problems above, SAM + suffix-link tree DP is also viable but requires extra bookkeeping. The suffix tree is the right choice when:

- You need to enumerate leaves in sorted order (the suffix array is essentially the DFS order of the tree).
- The problem involves subtree masks (multi-string problems).
- You need the LCA structure for $\mathcal{O}(1)$ LCP queries.

For other problems (occurrence counts, distinct substrings count, online substring tests), SAM is shorter.

27 Bioinformatics Applications

String algorithms are the foundation of modern bioinformatics. This chapter presents four problems adapted from bioinformatics practice, each reduced to a CP-style algorithm.

27.1 Read alignment with k-mismatch

Statement. Given a reference genome G ($\sim 10^6$) and many short reads $R_1, \ldots, R_m$ ($|R_i| \sim 100$), find for each R_i a position in G where it matches with at most k mismatches. (This is the "DNA read alignment" problem, simplified to exact matching with k errors.)

Reduction. For each read, use the Landau–Vishkin algorithm with the suffix array of G for LCP queries. Total: $\mathcal{O}(|G| \log n + \sum |R_i| \cdot k)$.

Reference. This is exactly what BWA and Bowtie do, modulo heuristics.

27.2 Variant calling: find positions where two genomes differ

Statement. Given two genomes G_1 and G_2 of similar length ($\sim 10^6$ each), enumerate all positions where they differ.

Reduction. Use hashing to find the LCP of G_1 and G_2 via binary search, then advance past the first mismatch, repeat. $\mathcal{O}(\text{mismatches} \cdot \log n)$.

Reference. This is the core of `diff` on DNA, used in modern variant callers.

27.3 de novo assembly: shortest common superstring

Statement. Given many short reads $R_1, \ldots, R_m$, find the shortest string containing each R_i as a substring. This is NP-hard in general, so we use a greedy heuristic.

Reduction. Greedy: repeatedly merge the two strings with the longest overlap. The overlap test is $\mathcal{O}(|R_i|)$ via hashing. Total: $\mathcal{O}(m^2 \cdot |R|)$, with $|R| \sim 100$ and $m \sim 10^4$.

Reference. This is the simplest "shortest common superstring" heuristic; modern assemblers (SPAdes, etc.) use de Bruijn graphs.

27.4 de Bruijn graph assembly

Statement. For $k = 31$ (typical), build the de Bruijn graph of G from its k-mers. Find an Eulerian path through this graph; the path spells a candidate genome.

Reduction. Build the de Bruijn graph: nodes are $(k-1)$-mers, edges are k-mers. Eulerian path via Hierholzer. $\mathcal{O}(|G|)$ total.

Reference. This is the core of `velvet`, `SOAPdenovo`, and other de Bruijn-graph assemblers.

27.5 When to use bioinformatics-style string algorithms

When to use

- Read alignment → Landau–Vishkin with SA for LCP.
- Variant calling → hashing + binary search.
- Shortest common superstring → greedy + hashing.
- de novo assembly → de Bruijn graph + Eulerian path.
- Modern assemblers combine all four with sophisticated heuristics; CP-style versions strip the heuristics for clarity.

27.6 Codeforces references

- **CF 1003F**: shortest common superstring with $k \leq 10$.
- **CF 81A**: Eulerian path on a graph (de Bruijn-like).

28 Bit-Parallel Algorithms: Worked Examples

This chapter presents four worked examples of bit-parallel algorithms that go beyond the basic Shift-And / Shift-Or / Myers of Chapter 11.

28.1 Example 1: Bit-parallel LCS

Statement. Compute the length of the LCS of two strings A and B with $|A| \leq 64$.

Algorithm. For each character c, precompute a bit-mask M_c where bit i is 1 iff $B[i] = c$. Maintain the DP "match-vector" V where bit i is 1 iff there's a common subsequence of $A[0..]$ and $B[0..i]$ of the current length. Update with the standard bit-parallel recurrence:

$$V' = (V + (V \,\&\, M_c)) \mid (V \,\&\, M_c) \mid V$$

(modulo carry handling). The LCS length is the position of the highest set bit in V after processing all of A. Total: $\mathcal{O}(|A|)$.

28.2 Example 2: Bit-parallel regex matching (Shift-Or on NFA)

Statement. Given a regex R of length m and a text T, check if R matches any substring of T.

Algorithm. Use Thompson's construction to build the NFA in $\mathcal{O}(m)$. The NFA has at most m states. Simulate it with bit-parallel Shift-Or: maintain a state bit-vector S where bit i is 1 iff NFA state i is active. For each character c of T, update S by the NFA's transition table encoded as a bit-mask. Total: $\mathcal{O}(m + |T| \cdot m/w)$.

28.3 Example 3: Bit-parallel palindrome substring detection

Statement. Given S ($|S| \leq 64$), enumerate all palindromic substrings.

Algorithm. For each center c (odd palindrome) or pair $(c, c+1)$ (even), use bitmasks to compare $S[c-k..c]$ with $\text{rev}(S[c..c+k])$ bit-parallel. The character comparison is a bit-vector XOR with a character mask. $\mathcal{O}(|S|^2/w)$.

28.4 Example 4: Bit-parallel subset enumeration on small alphabet

Statement. Given S over a small alphabet (say $\Sigma = \{a, b, c\}$) of length $n \leq 64$, enumerate all character-class subsets $\Sigma' \subseteq \Sigma$ such that S restricted to Σ' is a palindrome.

Algorithm. Enumerate all $2^{|\Sigma|}$ subsets; for each, build the bitmask of positions in S that survive (have a character in Σ'), check palindrome property via bit-parallel comparison. $\mathcal{O}(2^{|\Sigma|} \cdot n/w)$.

28.5 When to use bit-parallel examples

When to use

The common thread: a small string (≤ 64) and a problem whose DP can be encoded in bit-vectors. For each:

- LCS, edit distance, regex matching: use the standard bit-parallel DP.
- Substring / subsequence property checks: use character bit-masks.

- Multi-alphabet enumeration: combine with character-class reduction.

The threshold is $|S| \leq 64$. For larger inputs, fall back to standard algorithms.

29 Approximate Matching: Worked Examples

This chapter presents three worked approximate-matching problems beyond the basic Landau–Vishkin and Myers of Chapter 12.

29.1 Problem 1: Pattern with character classes and k-mismatch

Statement. Pattern P contains character classes (e.g., $P =$ `a[bc]d` meaning $P[1]$ can be b or c). Find positions in T where some assignment of P matches T with $\leq k$ mismatches.

Algorithm. Replace the per-character mask M_c with $M_{\text{class}} = \bigvee_{c \in \text{class}} M_c$. The bit-parallel k-mismatch update works unchanged with this generalisation.

29.2 Problem 2: Approximate pattern matching with gaps

Statement. Pattern P may match $T[i..j]$ if some subsequence of $T[i..j]$ equals P with bounded gaps.

Algorithm. DP over (text position, pattern position), with each pattern position having a "gap budget". Total: $\mathcal{O}(|T| \cdot |P|)$.

29.3 Problem 3: Edit distance with bounded workspace

Statement. Compute edit distance with $\mathcal{O}(\min(|A|, |B|))$ memory.

Algorithm. Use the rolling-row DP: only two rows of the DP table are kept; the answer is the last value of the second row. $\mathcal{O}(\min(|A|, |B|))$ memory, $\mathcal{O}(|A| \cdot |B|)$ time. For Myers' bit-vector, the memory is $\mathcal{O}(|B|/w)$.

29.4 When to use these variants

When to use

The variants are most useful when:

- The pattern has structure (character classes, optional positions, gaps).
- Memory is constrained.
- The text is very long and we need online / streaming approximate matching.

30 String Hashing with FFT and Convolution

For polynomial hashing with pattern matching over wildcards, the FFT gives an $\mathcal{O}(n \log n)$ solution where the standard algorithms would be $\mathcal{O}(n^2)$. This is the basis of the Clifford–Koch–Minkowski approach to pattern matching with wildcards.

30.1 Pattern matching with wildcards via FFT

Statement. Given T and P where some positions in P are wildcards (match any character), find all positions i in T such that P matches $T[i..i + |P| - 1]$.

Reduction. Define the match-score $m(i) = \sum_{j=0}^{|P|-1} [P[j] \neq ? \wedge T[i+j] \neq P[j]] \cdot 1$. A match at position i iff $m(i) = 0$. Rewrite $m(i)$ as a polynomial convolution and compute via FFT in $\mathcal{O}(n \log n)$ total.

Algorithm. Encode each character c as a polynomial $C_c(x) = \sum_{j:T[j]=c} x^j$. For each $c \in \Sigma$, compute the convolution with P's positions where c is required, and accumulate. The total mismatch count $m(i)$ is $\sum_c |P_c| \cdot |T_c| - 2 \cdot \text{conv}_c[i]$, where conv_c is the convolution. Total time: $\mathcal{O}(|\Sigma| \cdot n \log n)$.

30.2 Edit distance with wildcards via FFT

For edit distance where some characters are "don't cares", the FFT approach combines with bit-parallel DP to give $\mathcal{O}(n \log n + n \cdot k/w)$ time, where k is the error budget.

30.3 When to use FFT-based string algorithms

When to use

FFT-based string matching is the right tool when:

- The pattern has many wildcards and $|P| \leq n$.
- The Hamming / edit distance must be computed for many positions in parallel.
- The standard KMP / Z / hashing approaches are too slow.

The FFT constant factor is large; for $n \leq 10^5$, hashing + binary search is usually faster. FFT wins for $n \geq 10^6$ with many wildcards.

30.4 Codeforces references

- **CF 1190F**: FFT for geometric / string problems.
- **Emax blog on FFT**: canonical reference.

31 Multiple Pattern Matching: A Comparison

Multiple pattern matching is the problem of finding all occurrences of any of k patterns $P_1, \ldots, P_k$ in a single text T. Three algorithms compete: Aho–Corasick, suffix-array-with-binary-search, and hashing. We compare them.

31.1 Aho–Corasick

- Build: $\mathcal{O}(\sum |P_i| \cdot |\Sigma|)$.
- Match: $\mathcal{O}(|T| + \text{matches})$.
- Memory: $\mathcal{O}(\sum |P_i| \cdot |\Sigma|)$.
- Best when: patterns are given upfront and the text is single.

31.2 Suffix array + binary search per pattern

- Build: $\mathcal{O}(|T| \log |T|)$.
- Match: $\mathcal{O}(\sum |P_i| \log |T|)$.
- Memory: $\mathcal{O}(|T|)$.
- Best when: T is fixed and patterns arrive as queries, or when patterns are not given upfront.

31.3 Hashing + verification

- Build: $\mathcal{O}(|T|)$.
- Match: $\mathcal{O}(|T| + \sum |P_i|)$ with k-length hashing + hash set lookup.
- Memory: $\mathcal{O}(|T|)$.
- Best when: all patterns have the same length and the test is not adversarial.

31.4 Comparison and decision rule

When to use

- **All patterns same length, no adversarial tests**: hashing is fastest and shortest.
- **Many patterns of varying length, all given upfront**: Aho–Corasick.
- **Fixed text, patterns as queries**: suffix array + binary search.
- **Need streaming, single pattern**: failure automaton (KMP DFA).
- **Need streaming, many patterns**: Aho–Corasick (its online behaviour is one of its killer features).

32 Advanced Periodicity: Fine–Wilf and Beyond

Periodicity is the algebraic structure underlying much of string algorithms. This chapter collects the major theorems and their CP consequences.

32.1 Fine–Wilf theorem

Theorem 32.1 (Fine–Wilf 1965)**.** *If S of length n has periods p and q with $p+q-\gcd(p,q) \le n$, then S has period $\gcd(p,q)$. The bound is tight: there is a string of length $p+q-\gcd(p,q)-1$ with periods p, q and* not *period $\gcd(p,q)$.*

Proof sketch

The proof is by induction on $\max(p,q)$. Suppose $p < q$. For each $i \in [0, n-p-q]$, we have $S[i] = S[i+p] = S[i+q]$ by the two periodicity assumptions (the second equality requires $i+q < n$ and follows from the first period being shorter). Applying this argument recursively shows that characters at positions i and $i+\gcd(p,q)$ are equal, hence S has period $\gcd(p,q)$. The bound is tight because of the explicit counterexample: $S = (a^{p-1}b)^k$ where k is chosen so that $|S| = p+q-\gcd(p,q)-1$.

32.2 Periodicity lemma corollaries

Corollary 32.2 (Smallest period)**.** *The smallest period of S is $n - \pi[n-1]$ if this divides n, else n.*

Corollary 32.3 (Number of cyclic rotations equal to S)**.** *The number of distinct cyclic rotations of S is $n/\gcd(n,p)$ where p is the smallest period. The number of rotations equal to S itself is $\gcd(n,p)$.*

Corollary 32.4 (Periods of a string)**.** *If S has period p, it has period kp for every $k \ge 1$ with $kp \le n$.*

32.3 Border array and periodicity

The border array $b[i] = \pi[i]$ directly encodes the periodicity structure of S. The string $S[0..i]$ has period $i+1-\pi[i]$, and the sequence $\pi[i], \pi[\pi[i]-1], \ldots, 0$ enumerates all border lengths, equivalently all periods.

32.4 Critical factorisation theorem

Theorem 32.5 (Critical factorisation, Duval 1983)**.** *For any S, there exists a position f (called a "critical point") such that the order of left-vs-right comparison of characters $S[f-1]$ vs $S[f]$ in the rotations of S determines the lexicographically smallest rotation. The smallest rotation begins at position $f + \mathrm{LCP}(S[f..], S[..f-1])$.*

This theorem gives an alternative $\mathcal{O}(n)$ algorithm for the smallest cyclic rotation: find a critical point, then compute the LCP. In practice, Booth's algorithm is preferred for its simplicity.

32.5 When to use advanced periodicity

When to use

- Fine–Wilf gives the smallest period and primitive-word test in $\mathcal{O}(n)$.
- The border array gives all periods of S in $\mathcal{O}(n)$.
- The critical factorisation theorem is the theoretical basis for Booth's algorithm.
- For most CP problems, the periodicity lemma from Chapter 3 suffices; the advanced theorems are for IOI-medal level and research problems.

33 String Compression Algorithms

String compression is the algorithmic side of file formats like `gzip`, `bzip2`, and `zstd`. This chapter introduces the major compression algorithms and their CP-style implementations.

33.1 Run-length encoding (RLE)

Replace runs of identical characters with (c, k) pairs. `aaabbcccc` becomes `(a,3)(b,2)(c,3)`. Linear time, trivial to invert.

33.2 Lempel–Ziv–Welch (LZW)

Definition 33.1 (LZW)**.** LZW maintains a dictionary of seen strings, initially just the alphabet. At each step, find the longest prefix w of the remaining input that is in the dictionary; output w's code; insert $w+$ next character into the dictionary; advance past w.

LZW is the basis of `compress` and the GIF format. It achieves $\mathcal{O}(n)$ time and 50-70% compression on text. The dictionary is implicit in the encoded stream, so decompression needs no dictionary side-channel.

33.3 Burrows–Wheeler transform + Move-to-Front (BWT + MTF)

Apply BWT (Chapter 10); then move-to-front transform (each character replaced by its rank in a list, then moved to front); then run-length encoding or Huffman coding. This is the basis of `bzip2`.

33.4 RePair grammar compression

Replace each most-frequent bigram in S with a non-terminal; repeat. The result is a context-free grammar generating exactly $\{S\}$. The grammar is $\mathcal{O}(z \log n)$ in size where z is the LZ77 parse complexity.

33.5 LZ77 sliding-window

Definition 33.2 (LZ77)**.** At each position, find the longest match in the previous W characters (the sliding window); output (offset, length, next char).

The longest match is found by maintaining a suffix tree (or suffix automaton) of the window and walking it. Total time $\mathcal{O}(n)$ amortised. LZ77 is the basis of `gzip`, `zlib`, `zstd`, and `LZ4`.

33.6 When to use string compression

When to use

- For implementing `gzip`-style compression in CP: LZ77 + Huffman.
- For better compression: BWT + MTF + RLE + Huffman (the `bzip2` pipeline).
- For theoretical analysis of a string's compressibility: the LZ77 parse length z is the standard measure.
- For grammar-based compression: RePair is the simplest.

Compression problems are rare in CP but appear in ICPC-style "implement an encoder/decoder" problems.

33.7 Codeforces references

- **CF 1700F**: puzzle involving string transforms.
- **Emax blog on BWT**: includes the compression-pipeline view.

34 Squares, Repetitions, and Maximal Runs

A *square* is a substring of the form ww (e.g., `abab`). A *maximal repetition* (or *run*) is a maximal substring with a period $p \le |S|/2$. This chapter collects the structural theorems and algorithms.

34.1 Number of squares

Theorem 34.1 (Fraenkel–Simpson 1998)**.** *A string of length n has at most $2n$ distinct squares (substrings of the form ww).*

The bound was tightened to $2n - \Theta(\log n)$ by Ilie 2007. The conjectured tight bound is n.

34.2 Number of runs

Theorem 34.2 (Kolpakov–Kucherov 1999, runs theorem)**.** *A string of length n has at most n runs. The sum of exponents (the ratio of run length to period) over all runs is at most n.*

34.3 Algorithm: enumerate all runs in $\mathcal{O}(n)$

The full algorithm is the topic of a 100-page PhD thesis; in summary:

1. Compute the LZ-factorisation of S (each factor is the longest prefix seen so far, or one character if none).
2. Build the LCP array (Chapter 6).
3. For each potential period p (these are the divisors of n that are LCP values), test for runs of period p via LCP queries.

34.4 All squares via runs

Every square ww is contained in some run with period $|w|$. So enumerating all runs and then per-run enumerating the squares within gives all squares in $\mathcal{O}(n)$ time.

34.5 When to use squares / runs

When to use

- "Count distinct squares in S" → runs enumeration.
- "Find all periodic substrings of S with given period" → LCP array + sliding window.
- "Is S a power of T?" → periodicity lemma (Chapter 32).
- "Find all maximal repetitions" → runs theorem algorithm.

These problems are rare in CP and more common in research.

35 Suffix Automaton: Advanced Topics

This chapter collects advanced SAM topics not covered in the primer or Chapter 7.

35.1 SAM as the minimal DFA accepting suffixes

The SAM is the minimal DFA accepting the language $L_S = \{w : w \text{ is a suffix of } S\}$. This characterisation is the cleanest way to derive the SAM: start with the trivial DFA, apply Hopcroft minimisation, and you get the SAM.

35.2 Endpos tree topology

The suffix-link tree of the SAM is the *endpos tree*: each state's endpos set is a strict subset of its parent's. This tree has a natural parent array ('link[]') and is a tree of size $\leq 2n - 1$.

35.3 Substring DP on SAM

The general DP for substring problems:

- Each state v corresponds to a class of substrings (those spelled by paths from root to v).
- The size of the class is $\text{len}(v) - \text{len}(\text{link}(v))$.
- To count "substrings with property P", propagate $\text{has}(v)$ up the suffix-link tree; the count is $\sum_v (\text{len}(v) - \text{len}(\text{link}(v))) \cdot \text{has}(v)$.

35.4 Computing the smallest lexicographic substring with property P

DFS from the root, choosing the smallest available transition at each step, until we reach a state with property P. The path spells the answer.

35.5 Online SAM operations

The SAM supports online operations:

- **Append a character**: $\mathcal{O}(1)$ amortised.
- **Append a string**: $\mathcal{O}(|T|)$ amortised.
- **Append a suffix of the current SAM**: $\mathcal{O}(1)$ amortised, since the suffix is already in the SAM.

35.6 When to use SAM advanced topics

When to use

- For online substring-count problems with complex properties.
- For substring DP problems where the property is determined by endpos-class.
- For "smallest substring with property"-style problems.

36 Order Statistics on Strings

Many CP string problems ask for the "k-th lexicographically smallest" something. This chapter collects the techniques.

36.1 k-th lexicographically smallest suffix

Build the suffix array; the answer is $S[\text{sa}[k-1]..]$. $\mathcal{O}(n \log n)$ preprocessing, $\mathcal{O}(1)$ per query.

36.2 k-th lexicographically smallest substring

See Chapter 16: walk the suffix array, accumulating distinct substrings in sorted order, until we hit the k-th.

36.3 k-th lexicographically smallest cyclic rotation

Build the suffix array of $S+S$; the answer is the rotation starting at the k-th smallest suffix whose starting index is in $[0, n-1]$. Equivalently, use Duval's Lyndon factorisation.

36.4 k-th lexicographically smallest string of length n avoiding patterns

Build AC + matrix exponentiation; the k-th string corresponds to the k-th accepted path of length n in the AC automaton. This requires tracking the cumulative count at each state, then descending.

36.5 When to use order statistics

When to use

- "k-th suffix / substring" → SA + walk.
- "k-th rotation" → SA on $S + S$.
- "k-th string avoiding patterns" → AC + matrix + descending.

37 Trie Variants: PATRICIA, Compressed Trie, Radix Trie

The standard trie wastes a node per character. Three variants compress this redundancy, with different trade-offs.

37.1 PATRICIA trie

A PATRICIA (Practical Algorithm To Retrieve Information Coded In Alphanumeric) trie compresses unary paths: each internal node represents a substring of length > 1, stored as a (offset, length) pair into the original strings. PATRICIA tries have $O(k)$ nodes for k stored strings regardless of total length.

37.2 Compressed trie

Same idea, but the nodes are explicit (with full substring labels rather than offset+length). Useful for visualisation; equivalent in complexity.

37.3 Radix trie

A radix trie is a PATRICIA trie where each internal node has at most r children (a radix-r trie). For $r = 2$, this is a binary trie used for IP routing tables and XOR problems.

37.4 When to use trie variants

When to use

- Standard trie: short strings, small alphabet.
- PATRICIA / compressed trie: long strings, when memory is tight.
- Radix trie: when the alphabet is naturally bit-based (IP addresses, hashes) or when we need a specific branching factor.

38 Distributed and Parallel String Algorithms

When n exceeds the memory of a single machine ($n \geq 10^{10}$), or when wall-clock time matters more than total CPU time, we use distributed / parallel string algorithms. These are rare in CP but common in production.

38.1 Parallel suffix array construction

The SA-IS algorithm admits a parallel implementation: the LS-classification and bucket-sort can be parallelised across cores. For $n \sim 10^9$, a 16-core machine achieves a 12× speedup over single-core.

38.2 Distributed BWT via MapReduce

Build the suffix array in chunks of size B, sort the chunks in parallel, and merge. The BWT is then the last-column of the merged sorted rotations. Total: $\mathcal{O}(n \log n)$ work, $\mathcal{O}(n/P)$ wall-clock time on P cores.

38.3 Parallel Aho–Corasick

Distribute the text across P cores; each core runs AC on its chunk. The complication: patterns may straddle chunk boundaries. Solution: send the last $L-1$ characters of each chunk to the next core, where $L = \max |P_i|$.

38.4 When to use distributed / parallel

When to use

For typical CP problems ($n \leq 10^7$), single-core algorithms are sufficient. Distributed / parallel is for:

- Massive bioinformatics datasets ($n \geq 10^9$).
- Online services with high throughput requirements.
- Streaming architectures where the text never ends.

39 Open Problems and Research Directions

This chapter collects open problems in string algorithms. They are interesting research directions for the curious reader.

39.1 Tight bound on the number of distinct squares

Theorem 39.1 (Fraenkel–Simpson 1998)**.** *A string of length n has at most $2n$ distinct squares.*

The conjectured tight bound is n. The current best upper bound is $2n - \Theta(\log n)$ (Ilie 2007). Closing the gap is open.

39.2 Optimal suffix-array construction in the cache-aware model

The SA-IS algorithm is $\mathcal{O}(n)$ in time but performs poorly in cache (irregular memory access). Designing an algorithm that is $\mathcal{O}(n/B)$ cache-aware (where B is the cache line size) is open.

39.3 Compressed text indexing with constant edit operations

The FM-index supports count and locate in $\mathcal{O}(|P| \log \sigma)$ time. Is there an $\mathcal{O}(|P|)$ index in $\mathcal{O}(n \log \sigma)$ bits for arbitrary alphabets? Open.

39.4 Edit distance in truly subquadratic time

Theorem 39.2 (Backurs–Indyk 2015, conditional lower bound)**.** *If edit distance can be computed in time $O(n^{2-\varepsilon})$ for some $\varepsilon > 0$, then SETH (the Strong Exponential Time Hypothesis) is false.*

This conditional lower bound suggests that $\mathcal{O}(n^2/w)$ (Myers' bit-parallel) is essentially optimal for general edit distance. Bit-parallel with multi-word arithmetic achieves $\mathcal{O}(n^2/w^2)$ in some cases, but the gap to $\mathcal{O}(n^{2-\varepsilon})$ remains.

39.5 Hard problems in pattern avoidance

Counting strings of length n avoiding a set of patterns is $\mathcal{O}(|Q|^3 \log n)$ via matrix exponentiation. For $|Q| \sim 1000$ and $n \sim 10^{18}$, this is barely feasible. Faster algorithms (e.g., via polynomial matrices) exist but are not in the CP canon.

39.6 When to engage with open problems

When to use

- As a CP solver, you should know which problems are open: you cannot solve them in contest, but recognising them saves time.
- For research / Olympiad training: the open problems here are the next level after mastering this book.
- Reading the cited papers is the natural follow-up.

40 Complexity Reference Card

A condensed reference of every algorithm in this book with its time and space complexity.

40.1 Time complexities

Algorithm	Preprocessing	Per query	Space
KMP / Z search	$\mathcal{O}(\|P\|)$	$\mathcal{O}(\|T\|)$	$\mathcal{O}(\|P\|)$
Failure automaton	$\mathcal{O}(\|P\| \cdot \|\Sigma\|)$	$\mathcal{O}(1)$ per char	$\mathcal{O}(\|P\| \cdot \|\Sigma\|)$
Polynomial hashing (random base)	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Rabin–Karp matching	$\mathcal{O}(\|P\|)$	$\mathcal{O}(\|T\|)$ expected	$\mathcal{O}(\|P\|)$
Trie (insert / find / prefix count)	—	$\mathcal{O}(\|s\|)$ per op	$\mathcal{O}(\sum \|s_i\| \cdot \|\Sigma\|)$
Aho–Corasick build + match	$\mathcal{O}(\sum \|P_i\| \cdot \|\Sigma\|)$	$\mathcal{O}(\|T\| + \text{matches})$	$\mathcal{O}(\sum \|P_i\| \cdot \|\Sigma\|)$
AC + matrix exponentiation	$\mathcal{O}(\|Q\|^3)$ setup	$\mathcal{O}(\|Q\|^3 \log n)$ per query	$\mathcal{O}(\|Q\|^2)$
Suffix array (sort-by-prefix)	$\mathcal{O}(n \log^2 n)$	—	$\mathcal{O}(n)$
SA-IS	$\mathcal{O}(n)$	—	$\mathcal{O}(n)$
LCP via Kasai	$\mathcal{O}(n)$ after SA	—	$\mathcal{O}(n)$
LCP sparse table	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	$\mathcal{O}(n \log n)$
Suffix automaton	$\mathcal{O}(n)$ amortised	$\mathcal{O}(\|T\|)$ per substring test	$\mathcal{O}(n)$
Ukkonen suffix tree	$\mathcal{O}(n)$	$\mathcal{O}(\|P\|)$ per match	$\mathcal{O}(n)$
Generalised SAM	$\mathcal{O}(\sum \|S_i\|)$	$\mathcal{O}(\|T\|)$ per query	$\mathcal{O}(\sum \|S_i\|)$
BWT + FM-index + wavelet	$\mathcal{O}(n \log n)$	$\mathcal{O}(\|P\| \log \sigma)$ per match	$\mathcal{O}(n \log \sigma)$ bits
Shift-Or exact matching	$\mathcal{O}(\|P\| \cdot \sigma)$	$\mathcal{O}(\|T\|)$	$\mathcal{O}(\|P\| \cdot \sigma / w)$
Myers bit-vector edit distance	$\mathcal{O}(\|B\| \cdot \sigma)$	$\mathcal{O}(\|A\|)$	$\mathcal{O}(\|B\|)$
Bit-parallel k-mismatch	$\mathcal{O}(\|P\| \cdot \sigma)$	$\mathcal{O}(\|T\|)$	$\mathcal{O}(\|P\| \cdot k / w)$
Landau–Vishkin k-difference	$\mathcal{O}(\|T\| \log \|T\|)$ for LCP	$\mathcal{O}(\|T\| \cdot k)$	$\mathcal{O}(\|T\|)$
Booth / Duval rotation	—	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Eertree + series links	$\mathcal{O}(n)$ amortised	$\mathcal{O}(n \log n)$ factorisation	$\mathcal{O}(n)$

Algorithm	Preprocessing	Per query	Space
de Bruijn sequence	$\mathcal{O}(\sigma^k)$	—	$\mathcal{O}(\sigma^k)$
Wavelet tree	$\mathcal{O}(n \log \sigma)$	$\mathcal{O}(\log \sigma)$ per rank/select	$\mathcal{O}(n \log \sigma)$ bits

40.2 Space complexities in perspective

- Plain suffix array: $4n$ bytes (one int per index).
- Sparse-table LCP: $4n \log n$ bytes; ~ 80 MB for $n = 10^6$.
- Suffix automaton: $16n$ bytes with array transitions on $|\Sigma| = 26$; ~ 16 MB for $n = 10^6$.
- Suffix tree (Ukkonen): $\sim 30n$ bytes with map transitions.
- BWT + FM-index + wavelet: $\sim 1.5n$ bytes total.
- For $n \geq 10^7$, BWT/FM is the only feasible structure.

40.3 Heuristic for choosing algorithm

When to use

- $n \leq 10^5$, simple problem: hashing or trie.
- $n \leq 10^6$, substring problems: SA + LCP.
- $n \leq 10^6$, online substring problems: SAM.
- $n \leq 10^7$, large alphabet: SAM with map transitions.
- $n \geq 10^7$, small alphabet: BWT + FM-index.
- Many patterns, short text: AC.
- Few patterns, long text: SA + binary search per pattern.
- Edit distance, one string ≤ 64: Myers bit-vector.
- Edit distance, both strings large: standard $\mathcal{O}(nm)$ DP.

41 Common Pitfalls and Debugging String Algorithms

This chapter collects the 12 most common bugs in CP string code. For each, we describe the bug, give the symptom, and prescribe the fix.

41.1 Pitfall 1: Off-by-one in substring hash extraction

Bug. The formula $H(S[l..r]) = h[r+1] - h[l] \cdot b^{r-l+1} \bmod p$ is applied with $r+1$ replaced by r. The symptom is that hashes of length-1 substrings are wrong.

Fix. Always use $r+1$ for the right index. Add a unit test for $S = \texttt{a}$ and $l = r = 0$.

41.2 Pitfall 2: Overflow in polynomial hash multiplication

Bug. $h[l] \cdot b^{r-l+1}$ overflows a 64-bit integer when both factors are $\sim 10^{18}$. The result silently wraps to the wrong value.

Fix. Use `__int128` for the multiplication, or use the Mersenne reduction trick for $p = 2^{61} - 1$.

41.3 Pitfall 3: Forgotten separator in $P\#T$

Bug. Without a separator, the Z function or KMP π array of $P + T$ may match positions where P extends into T, producing false positives.

Fix. Always use a separator character (e.g., `#` or `\0`) that does not occur in P or T.

41.4 Pitfall 4: Suffix array construction with `std::sort` is too slow

Bug. The $\mathcal{O}(n \log^2 n)$ SA with `std::sort` works for $n \leq 10^5$ but TLEs on $n \geq 10^6$.

Fix. Replace `std::sort` with radix sort (for the $\mathcal{O}(n \log n)$ version), or implement SA-IS for $n \geq 10^6$.

41.5 Pitfall 5: SAM map-based transitions are too slow for $|\Sigma| = 26$

Bug. Using `std::map<char, int>` for SAM transitions is $\mathcal{O}(\log 26)$ per operation, slowing SAM from $\mathcal{O}(n)$ to $\mathcal{O}(n \log 26)$ with a constant factor of ~ 5.

Fix. For $|\Sigma| \leq 26$, use `int next[26]` array; for larger alphabets, use `std::unordered_map` (faster than `std::map` on average).

41.6 Pitfall 6: Eertree's two roots are confused

Bug. The eertree has two roots: state 0 (length 0, the empty palindrome) and state 1 (length -1, the imaginary). Forgetting to set up state 1 with `link = 1` (self) causes infinite recursion in `getLink`.

Fix. Always initialise state 1 with `link = 1` (it links to itself). Verify with a unit test on $S =$ `aa` (a single-char palindrome followed by the same char).

41.7 Pitfall 7: BWT construction uses the wrong sentinel

Bug. Using `\0` as a sentinel in C++ strings truncates them at the first occurrence. The BWT construction then has missing suffixes.

Fix. Use `\1` or another low-but-nonzero byte as the sentinel.

41.8 Pitfall 8: Myers bit-vector edit distance with negative numbers

Bug. The Myers DP requires unsigned arithmetic, but the carry-chain assumes signed semantics for the horizontal delta. Using signed integers causes the carry to wrap incorrectly.

Fix. Use `uint64_t` throughout, with explicit XOR-based carry handling. Test on $A = B =$ `aaaa` first (the edit distance is 0).

41.9 Pitfall 9: Aho–Corasick output links missed

Bug. The AC output links should be followed for every state visited during matching, not just the terminal states. Forgetting this misses patterns that end at `fail` or `fail.fail` states.

Fix. In the match loop, walk `u = v; while u:  if u is terminal, report; u = fail[u]`.

41.10 Pitfall 10: Booth's algorithm with `s + s` doubling

Bug. Building `s2 = s + s` and using `s2[k + i + 1]` may index past the end if k and i are not bounded properly.

Fix. Pre-compute `s2 = s + s` as a separate string of length $2n$; all indices are then in $[0, 2n-1]$. Use `s2[(k + i + 1) % (2*n)]` for safety.

41.11 Pitfall 11: SA-IS recursion does not reduce alphabet size

Bug. The recursive SA-IS call must reduce the alphabet to a permutation of $\{0, \dots, k-1\}$ where k is the number of distinct LS-positions. If the alphabet is not reduced, the recursion does not terminate.

Fix. After classifying LS-positions, sort them and assign new ranks in $\{0, \dots, k-1\}$. Recurse on the renamed sequence.

41.12 Pitfall 12: Series-link eertree DP visits states in wrong order

Bug. The series-link DP requires processing states in length-increasing order. Processing in extension order (BFS) gives wrong answers because later states depend on series-link ancestors.

Fix. Sort states by `len` after extension; process DP in this order.

41.13 Debugging checklist

When to use

When your string code fails:

1. Test on small inputs: $S =$ `a`, `aa`, `ab`, `aba`.
2. Test on the "lazy" alphabet: $S =$ `aaaa...a` (catches edge cases in failure-link machinery).
3. Test on the adversarial alphabet: $S =$ `abcabcabc...` (catches periodicity bugs).
4. Print the structure (SA, π, Z, sa, lcp) and compare with hand-computed values.
5. Time the implementation against a stress test of $n = 10^6$ random characters.

42 Affine Gap and Advanced Edit Distance

The standard edit distance charges 1 per gap (insertion or deletion). Affine gap distance charges a fixed "open" cost for starting a gap plus a per-character "extend" cost. This better models biological evolution (a single insertion of k characters is one event, not k).

42.1 Affine gap definition

Definition 42.1 (Affine gap)**.** The cost of an insertion or deletion of k consecutive characters is $a + b \cdot k$, where a is the "open" penalty and b is the "extend" penalty.

42.2 Gotoh's algorithm

Theorem 42.2 (Gotoh 1982)**.** *The affine gap edit distance can be computed in $\mathcal{O}(|A| \cdot |B|)$ time using three DP tables: M (match/sub), I (insertion gap), D (deletion gap).*

$$M[i][j] = \min(M[i-1][j-1] + [A[i] \neq B[j]],\ I[i-1][j-1] + [A[i] \neq B[j]],\ D[i-1][j-1] + [A[i] \neq B[j]]),$$
$$I[i][j] = \min(M[i][j-1] + a + b,\ I[i][j-1] + b),$$
$$D[i][j] = \min(M[i-1][j] + a + b,\ D[i-1][j] + b).$$

The answer is $\min(M[n][m], I[n][m], D[n][m])$.

42.3 Bit-parallel affine gap

For $|B| \leq 64$, the Gotoh DP can be parallelised using Myers' trick applied to the three tables independently. Total: $\mathcal{O}(|A|)$ time.

42.4 When to use affine gap

When to use

- Biological sequence alignment where single-event insertions are common (DNA, protein).
- When the problem statement specifies gap-open and gap-extend penalties separately.
- For OCR / speech recognition where word-boundaries have special costs.

43 Online Suffix Tree Updates

Ukkonen's algorithm is online: it processes characters one at a time. But what about more general updates appending, deleting, replacing?

43.1 Appending a character

This is Ukkonen's main mode: $\mathcal{O}(1)$ amortised per character. The active point advances, suffix links are created lazily, and Rule 1/2/3 are applied as described in Chapter 8.

43.2 Prepending a character

This is harder: adding a character at the start of S invalidates the existing suffix tree's structure (every suffix changes). The naive solution is to rebuild, $\mathcal{O}(n)$ per prepend. There is a clever "reverse suffix automaton" construction that supports prepending in $\mathcal{O}(1)$ amortised, but it is rare in CP.

43.3 Replacing a character

Single-character point updates on the suffix tree are $\mathcal{O}(n)$ in the worst case (any suffix containing the modified character changes). In practice, we accept the rebuild cost or use a treap-based dynamic string structure with hashing (Chapter 23).

43.4 Deleting a character

Equivalent to a point update; same complexity as replacement.

43.5 When to use online updates

When to use

- Append-only streams: Ukkonen.
- Point updates: treap + hashing, accept $\mathcal{O}(\log n)$ per op.
- For non-append updates on suffix trees, the overhead of the clever constructions usually outweighs a rebuild for $n \leq 10^5$.

44 Worked Examples in Detail: Three Codeforces Problems

This chapter presents three Codeforces problems with full solutions: pattern recognition, algorithm choice, code, and walkthrough. The aim is to demonstrate the end-to-end reasoning from problem statement to AC submission.

44.1 Example 1: CF 1202E — You Are Given Some Strings

Statement. Given a string T of length n and a set $\mathcal{S}$ of k strings, count the number of pairs $(A, B) \in \mathcal{S}^2$ such that $A + B$ is a substring of T.

Pattern recognition. "For each pair of strings, check if their concatenation is a substring of T" $\rightarrow$ we need both "forward" matching (suffixes of T that begin with A) and "backward" matching (prefixes of T that end with B, i.e., suffixes of rev(T) that begin with rev(B)). This is the canonical AC + DP hybrid.

Algorithm.

1. Build an AC automaton on $\mathcal{S}$ (forward).
2. Build an AC automaton on $\{\text{rev}(s) : s \in \mathcal{S}\}$ (reverse).
3. For each position $i \in 0..n-1$, compute $f[i]$ = number of $A \in \mathcal{S}$ such that A matches T ending at position i (using the forward AC).
4. For each position i, compute $g[i]$ = number of $B \in \mathcal{S}$ such that rev(B) matches rev(T) ending at position i (using the reverse AC), i.e., B matches T starting at position $i + 1$.
5. The answer is $\sum_i f[i] \cdot g[i]$.

Complexity. $\mathcal{O}(|T| + \sum |s_i|)$ time, $\mathcal{O}(\sum |s_i|)$ memory.

CF 1202E solution sketch

```cpp
// Forward AC: walk T, for each position compute f[i] = # of patterns ending at i.
// Reverse AC: walk rev(T), for each position compute g[i] = # of patterns
// (rev) ending at i.  In rev(T), position i corresponds to position n-1-i in T.
// So g[n-1-i] = # of B whose rev matches rev(T) ending at i,
// which means B starts at position i+1 in T.
// Answer = sum over i of f[i] * g[n-2-i].
long long solve(const std::string& T, const std::vector<std::string>& S) {
    AhoCorasick fw, rv;
    for (int i = 0; i < (int)S.size(); ++i) {
        fw.insert(S[i], i);
        std::string r(S[i].rbegin(), S[i].rend());
        rv.insert(r, i);
    }
    fw.build(); rv.build();
    auto fwdMatch = fw.match(T);
    std::string rT(T.rbegin(), T.rend());
    auto revMatch = rv.match(rT);
    int n = T.size();
    std::vector<long long> f(n), g(n);
    for (int i = 0; i < n; ++i) for (int idx : fwdMatch[i]) ++f[i];
    for (int i = 0; i < n; ++i) for (int idx : revMatch[i]) ++g[n-1-i];
    long long ans = 0;
    for (int i = 0; i + 1 < n; ++i) ans += f[i] * g[i+1];
    return ans;
}
```

44.2 Example 2: CF 932G — Palindrome Partition

Statement. Given S of length n, partition it into the minimum number of even-length palindromes (after first rephrasing S as $s_0 s_{n-1} s_1 s_{n-2} \dots$).

Pattern recognition. "Minimum partition into palindromic substrings" → eertree + series links + DP. The rephrasing ensures that every even-length palindromic substring of the original S corresponds to a palindromic substring of the rephrased string.

Algorithm.

1. Rephrase: $T = s_0 s_{n-1} s_1 s_{n-2} \dots s_{(n-1)/2} s_{n/2}$. For $S =$ `abba`, $T =$ `aabb` (interleaving from both ends).
2. Build an eertree on T with series links.
3. DP: $\mathrm{dp}[i]$ = minimum number of palindromic factors of $T[0..i]$. Walk the series-link chain to compute $\mathrm{dp}[i] = \min_v \mathrm{dp}[i - \mathrm{len}(v)] + 1$ in $\mathcal{O}(\log n)$ per position.
4. Return $\mathrm{dp}[n-1]$.

Complexity. $\mathcal{O}(n \log n)$ time, $\mathcal{O}(n)$ memory.

44.3 Example 3: CF 235C — Cyclical Quest

Statement. Given S and queries Q_i, count the number of positions p in S such that some cyclic rotation of Q_i is a substring of S starting at p.

Pattern recognition. "Cyclic rotations of Q_i" → treat as $Q_i + Q_i$ and search for length-$|Q_i|$ substrings. "Count occurrences in S" → SAM on S.

Algorithm.

1. Build SAM on S.
2. For each query Q of length m:
3. Walk through $Q + Q[0..m-2]$ (length $2m-1$) in the SAM, tracking the current state and matched length l (capped at m).
4. At each step, if $l = m$, the current state v has $|\mathrm{endpos}(v)|$ occurrences. Add this count (avoiding double-counting for rotations that appear multiple times).

Complexity. $\mathcal{O}(|S| + \sum |Q_i|)$ preprocessing + query.

44.4 Lessons from the three examples

When to use

- CF 1202E: AC + DP. The two AC automata encode the two ends of the substring $A + B$.
- CF 932G: Eertree + series-link DP. The rephrasing trick converts even-length palindromes to general palindromes.
- CF 235C: SAM + cyclic walk. The cyclic structure is encoded by walking $2|Q| - 1$ characters with a cap on match length.

Each example demonstrates how a hybrid pattern reduces the problem to a known algorithm.

45 Pattern Recognition Drill: Twenty Quick-Fire Problems

This chapter is a self-quiz: read the problem statement, identify the algorithm, and check against the answer. Twenty problems, one algorithm each.

P1. "Count distinct substrings of S" → SA + LCP or SAM.

P2. "Longest common substring of two strings" → SAM on S_1 and walk S_2.

P3. "Number of strings of length n avoiding $\{P_1, \ldots, P_k\}$" → AC + matrix exponentiation.

P4. "Smallest cyclic rotation of S" → Booth or Duval.

P5. "Longest palindromic substring" → Manacher.

P6. "Minimum palindromic factorisation" → Eertree + series links.

P7. "All occurrences of P in T, $|P| \le 64$" → Shift-Or bit-parallel.

P8. "Edit distance of A and B, $|B| \le 64$" → Myers bit-vector.

P9. "Hamming-distance-k matching, $|P| \le 64$" → Bit-parallel k-mismatch.

P10. "Edit-distance-k matching, $|P| > 64$" → Landau–Vishkin.

P11. "Online substring-existence queries on a growing S" → Suffix automaton.

P12. "Online palindromic-substring queries" → Eertree.

P13. "For each T-query, number of occurrences in S" → Suffix array + binary search.

P14. "Maximum XOR pair in array A" → XOR trie.

P15. "k-th smallest element in $A[l..r]$" → Persistent trie.

P16. "Smallest lexicographic suffix of S" → Duval (last Lyndon factor).

P17. "Count occurrences of each P_i in T" → Aho–Corasick.

P18. "Is S a power of a shorter string?" → Prefix function periodicity check.

P19. "Number of distinct substrings of S that occur in T" → SAM on T, walk S.

P20. "Shortest string containing all P_i" → AC + TSP-DP.

45.1 How to use the drill

Read each problem, decide on an algorithm, then verify with the answer. If you missed one, re-read the relevant chapter. After working through the drill twice, you should be able to recognise these signatures instantly.

46 Comparative Survey: SAM vs SA vs Suffix Tree

Three suffix structures dominate CP: the suffix automaton (SAM), the suffix array (SA) + LCP, and the suffix tree (Ukkonen). They all encode the same information about S, but with different trade-offs. This chapter compares them head-to-head.

46.1 Size and construction cost

Structure	Size	Construction	Per query
SAM	$2n$ states, $3n$ transitions	$\mathcal{O}(n)$ amortised	$\mathcal{O}(\|T\|)$
SA + LCP	$2n$ integers	$\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$ SA-IS	$\mathcal{O}(\|P\| \log n)$ binary search, or $\mathcal{O}(\|P\| + \log n)$ with sparse table

Structure	Size	Construction	Per query
Suffix tree (Ukkonen)	$2n$ nodes, $2n$ edges	$\mathcal{O}(n)$ amortised	$\mathcal{O}(\|P\|)$ descent

46.2 Memory usage

- SAM: 4 bytes per state (assuming $|\Sigma| \le 26$ array transitions) + 4 bytes per `link` + 4 bytes per `len` $= \sim 12$ bytes $\times 2n = 24n$ bytes.
- SA + LCP + sparse table: $4n$ (SA) $+4n$ (LCP) $+4n \log n$ (sparse table) $= \sim 4n \log n + 8n$ bytes. For $n = 10^6$, ~ 88 MB.
- Suffix tree: $\sim 30n$ bytes with map transitions and explicit children.

46.3 What each structure is best at

- **SAM**: online substring-existence queries, online substring-count DP, cyclic-shift queries. The killer feature is $\mathcal{O}(1)$ amortised extension.
- **SA + LCP**: offline substring comparison, k-th substring queries, longest common substring, smallest cyclic shift. The killer feature is the sorted order of suffixes.
- **Suffix tree**: LCA-based $\mathcal{O}(1)$ LCP, subtree-leaf enumeration, multi-string common substring via subtree masks. The killer feature is the explicit tree structure.

46.4 What each is worst at

- **SAM**: enumerating suffixes in sorted order requires an extra DFS with sorting. The link tree is not sorted.
- **SA**: online updates require rebuilding; not streaming-friendly.
- **Suffix tree**: implementation is 200–300 lines of C++ and notoriously bug-prone.

46.5 Decision rule

When to use

- **Online** (characters stream in) → SAM or Ukkonen.
- **Offline, substring comparison queries** → SA + LCP + sparse table.
- **Offline, k-th something queries** → SA.
- **Need subtree structure** (multi-string problems, occurrence enumeration) → Ukkonen.
- **Need to count / DP on substrings with complex property** → SAM + suffix-link tree DP.
- **Memory is tight** ($n \ge 10^7$) → BWT + FM-index instead of any of these.

47 Implementation Tips and CP Contest Strategy

This chapter collects practical advice for coding string algorithms under contest conditions: which templates to memorise, how to debug, and how to choose the simplest structure that works.

47.1 What to have in your template file

A typical CP template file for strings should include:

1. Polynomial hashing with random base and double modulus (~ 30 lines).
2. Prefix function π and KMP search (~ 20 lines).
3. Z-function (~ 15 lines).
4. Aho–Corasick with array transitions (~ 60 lines).
5. Suffix array $\mathcal{O}(n \log^2 n)$ + Kasai LCP (~ 30 lines).
6. Suffix automaton with array transitions (~ 50 lines).
7. Sparse table for RMQ (~ 20 lines).
8. Trie and persistent trie (~ 40 lines).

Total: ~ 265 lines of pre-tested, debugged code. This is your "CP string toolbox"; you should know each piece's interface by heart.

47.2 Order of attempts

When solving a CP string problem, try in this order:

1. Read the problem statement carefully; identify the pattern (Chapter 52 drill).
2. Try hashing first (shortest, fastest to code).
3. If hashing is too slow or the test is adversarial, fall back to SAM.
4. If SAM does not fit the problem naturally, try SA + LCP.
5. If the problem is multi-pattern, use Aho–Corasick.
6. If the problem is palindromic, use eertree / Manacher.
7. If the problem is about cyclic rotations, use Booth / Duval.
8. If the problem requires bit-parallelism (small pattern, many queries), use Shift-Or / Myers.

47.3 Stress testing

Before submitting, stress-test on:

- Random strings of length 10, 100, 1000, 10000.
- The "lazy" alphabet: $S = \mathtt{a}^n$.
- The "alternating" alphabet: $S = \mathtt{abab}\ldots$.
- The "adversarial" alphabet: $S = \mathtt{aaaab}$ (catches failure-link bugs).
- Edge cases: $n = 1$, $n = 2$, $S = \varepsilon$.

47.4 Time-limit management

For $n \le 10^5$, almost any algorithm works. For $n = 10^6$:

- Hashing: ~ 0.1 s. Use it.
- KMP/Z: ~ 0.05 s. Use it.
- SA + LCP + sparse table: ~ 1 s. Use it.
- SAM: ~ 0.3 s. Use it.

- Ukkonen: ~ 1 s. Use it.
- AC + matrix exponentiation: ~ 0.1 s for $|Q| = 100$.

For $n \geq 10^7$: only BWT/FM-index and SAM with map transitions fit.

47.5 Common time-limit pitfalls

- Using `std::map` for SAM transitions when $|\Sigma| \leq 26$ wastes 5x.
- Building SA with `std::sort` when $n = 10^6$ TLEs.
- Using a naive $\mathcal{O}(n^2)$ DP for edit distance when $n = 1000$ is borderline.
- Forgetting to release memory between test cases (causes MLE).

48 Suffix-Link Tree Heavy-Light Decomposition

The suffix-link tree of the SAM is a real tree with $\leq 2n - 1$ nodes, and so it admits heavy-light decomposition (HLD). This is rare in CP but appears in some expert problems.

48.1 Why HLD on the suffix-link tree?

Consider the problem "for each prefix of S, count the number of palindromic substrings ending at that prefix." The naive DP uses the eertree's link tree, but for problems that involve longer-range queries (e.g., "for each prefix, the minimum number of distinct substrings that cover it"), the suffix-link tree of the SAM is the substrate, and queries may span $O(\log n)$ heavy paths.

48.2 Heavy-light decomposition

For a tree with n nodes, HLD partitions edges into "heavy" (the edge to the largest subtree) and "light". The heavy paths form a forest; each light jump at least halves the subtree size. A query from root to leaf crosses $O(\log n)$ heavy paths.

48.3 Application: per-prefix distinct substring count with constraints

Statement. For each prefix $S[0..i]$, count the number of distinct substrings ending at position i that also satisfy some property P.

Algorithm. Build the SAM. For each state, maintain a Fenwick tree over the heavy paths. Updates along the suffix-link chain are $O(\log^2 n)$ per query.

48.4 When to use HLD on SAM

When to use

- For per-prefix substring DP with non-trivial property predicates.
- For problems that ask "for each prefix, the answer to a tree-query on the suffix-link tree".
- The constant factor is high; prefer simpler structures when possible.

49 Epilogue: From CP to Research

You have reached the end of the expert edition. The toolbox now in your hands covers every major string-algorithm structure invented through 2025: hashing (with randomised bases and anti-collision theory), the prefix and Z functions (with periodicity lemmas and Fine–Wilf), the

trie and Aho–Corasick (with matrix exponentiation for counting), the suffix array (with SA-IS linear construction and sparse-table RMQ), the suffix automaton (with the suffix-link tree DP and the generalised multi-string SAM), Ukkonen's online suffix tree, the Burrows–Wheeler transform and FM-index (with wavelet trees), bit-parallel pattern matching (Shift-And/Or, BNDM, Myers bit-vector), approximate matching with k errors (Landau–Vishkin and the bit-parallel variant), the eertree with series links for palindromic factorisation, runs enumeration, and the Lyndon/Duval/Booth machinery for cyclic rotations.

49.1 What you should now be able to do

After working through the chapters and drills:

- Recognise the hybrid pattern of a CP string problem within 30 seconds.
- Implement any of the algorithms in this book from a template within 30 minutes.
- Diagnose the 12 most common bugs from Chapter 41 and fix them.
- Choose the optimal algorithm based on n, alphabet size, online/offline, and adversary.

49.2 Where to go from here

The next levels are:

1. **Original papers**: read the foundational papers in the annotated bibliography. Each is 10–20 pages and rewards multiple readings.
2. **Recent CPM / SPIRE / SODA proceedings**: the latest string-algorithms research; download proceedings from stringology.org and dblp.org/db/conf/cpm.
3. **Implement in a real project**: contribute to `sdsl-lite` (succinct data structures library) or `bwa` (bioinformatics aligner) to see these algorithms in production.
4. **Compete at IOI / ICPC World Finals level**: the hybrid-pattern catalogue here is the toolbox; the missing piece is contest experience.

49.3 Final thought

String algorithms are the cleanest corner of theoretical computer science: every structure has a clean complexity, every algorithm has a tight analysis, and every CP problem is recognisable. Master them, and a substantial fraction of all CP problems become a matter of pattern recognition. Good luck.

50 Glossary of String Algorithm Terms

This glossary collects the technical terms used throughout the book. Each entry includes a one-sentence definition and a pointer to the chapter where it is developed.

50.1 A–E

- **Aho–Corasick automaton**: DFA accepting all strings that contain any of k patterns as a substring; built by adding failure links to a trie of the patterns. See Ch. 5.
- **Active point**: the $(\text{node}, \text{edge}, \text{length})$ triple maintained by Ukkonen's algorithm to track the current suffix being extended. See Ch. 8.
- **Affine gap**: a gap cost of the form $a + b \cdot k$ for a gap of length k; computed via Gotoh's three-table DP. See Ch. 42.

- **BNDM**: Backward Nondeterministic Dawg Matching; the bit-parallel variant of Boyer–Moore. See Ch. 11.
- **Booth's algorithm**: $\mathcal{O}(n)$ algorithm for the lexicographically smallest cyclic rotation of S. See Ch. 15.
- **Border**: a proper prefix of S that is also a suffix. The border array $b[i] = \pi[i]$. See Ch. 3.
- **Burrows–Wheeler transform (BWT)**: the last column of the matrix of sorted cyclic rotations of $S\$$; reversible and compression-friendly. See Ch. 10.
- **de Bruijn sequence**: a cyclic string in which every length-k string over Σ appears exactly once. See Ch. 18.
- **Damerau–Levenshtein distance**: edit distance with an additional transposition operation. See Ch. 22.
- **Eertree**: palindromic tree; a DFA accepting exactly the palindromic suffixes of S at every prefix. See primer and Ch. 13.
- **Endpos equivalence**: two substrings are equivalent if their endpos sets are equal; the basis for SAM states. See Ch. 7.

50.2 F–N

- **Failure automaton**: the DFA version of KMP; precomputes all transitions for $\mathcal{O}(1)$ per text character. See Ch. 3.
- **Failure link**: in AC, a pointer from each trie node to the node representing the longest proper suffix that is also a trie prefix. See Ch. 5.
- **Fine–Wilf theorem**: if S has periods p, q with $p + q - \gcd(p, q) \leq n$, then S has period $\gcd(p, q)$. See Ch. 32.
- **FM-index**: Ferragina–Manzini compressed full-text index, based on BWT + LF-mapping + wavelet tree. See Ch. 10.
- **Generalised suffix automaton (GSAM)**: SAM extended to a set of strings, with per-string endpos masks. See Ch. 9.
- **Gotoh's algorithm**: $\mathcal{O}(|A| \cdot |B|)$ DP for affine gap edit distance. See Ch. 42.
- **KMP search**: pattern matching via the prefix function π on $P\#T$; $\mathcal{O}(|T| + |P|)$. See primer and Ch. 3.
- **Kasai's algorithm**: $\mathcal{O}(n)$ LCP array construction from the suffix array. See Ch. 6.
- **Landau–Vishkin algorithm**: $\mathcal{O}(|T| \cdot k)$ algorithm for k-error pattern matching. See Ch. 12.
- **Levenshtein distance**: the standard edit distance with insertions, deletions, substitutions. See Ch. 22.
- **LF-mapping**: the bijection from BWT rows to first-column rows; the basis of FM-index reversal. See Ch. 10.
- **Lyndon word**: a string strictly smaller than all its proper rotations. See Ch. 15.
- **Manacher's algorithm**: $\mathcal{O}(n)$ longest palindromic substring via transformed-string Z-style trick. See primer.
- **Myers' bit-vector**: $\mathcal{O}(|A|)$ edit distance for $|B| \leq 64$ using bit-parallel DP. See Ch. 11.
- **Necklace**: an equivalence class of strings under cyclic rotation. See Ch. 18.

50.3 P–Z

- **PATRICIA trie**: a trie with compressed unary paths; $\mathcal{O}(k)$ nodes for k stored strings. See Ch. 37.
- **Periodicity lemma**: if S has periods p, q with $p+q \le n+\gcd(p,q)$, then S has period $\gcd(p,q)$. See Ch. 32.
- **Persistent trie**: a trie that supports queries on previous versions, used for k-th smallest in range. See Ch. 14.
- **Prefix function** π: the array $\pi[i]$ = longest proper prefix-suffix of $S[0..i]$. See primer and Ch. 3.
- **Primitive word**: a string S that is not a power of a shorter string. See Ch. 18.
- **Rabin–Karp**: pattern matching via polynomial hashing; $\mathcal{O}(|T|+|P|)$ expected. See primer.
- **Run**: a maximal periodic substring with period $p \le |S|/2$. See Ch. 34.
- **Runs theorem**: a string of length n has at most n runs. See Ch. 34.
- **SA-IS**: $\mathcal{O}(n)$ suffix array construction by induced sorting of LS-positions. See Ch. 6.
- **Series link**: the eertree's compressed suffix link used for $\mathcal{O}(n \log n)$ palindromic factorisation. See Ch. 13.
- **Shift-And / Shift-Or**: bit-parallel exact pattern matching for $|P| \le 64$. See Ch. 11.
- **Sparse table**: $\mathcal{O}(n \log n)$ preprocessing, $\mathcal{O}(1)$ RMQ structure. See Ch. 6.
- **Square**: a substring of the form ww. See Ch. 34.
- **Suffix array (SA)**: the permutation sa such that suffixes are in lexicographic order. See Ch. 6.
- **Suffix automaton (SAM)**: the minimal DFA accepting all suffixes of S; $\le 2n-1$ states. See Ch. 7.
- **Suffix link**: in SAM, a pointer from each state to the state representing the longest proper palindromic / endpos-equivalent suffix. See Ch. 7.
- **Suffix tree**: the compacted trie of all suffixes of $S\$$; built online by Ukkonen in $\mathcal{O}(n)$. See Ch. 8.
- **Thue–Morse sequence**: the canonical hash-breaking sequence; parity of popcount in binary representation of i. See Ch. 2.
- **Trie**: a rooted tree where each path spells a stored string. See primer.
- **Ukkonen's algorithm**: $\mathcal{O}(n)$ online suffix tree construction via active-point and suffix links. See Ch. 8.
- **Wavelet tree**: a binary tree of bit-vectors supporting rank / select on a string in $\mathcal{O}(\log \sigma)$. See Ch. 21.
- **Z-function**: $z[i]$ = LCP of S and $S[i..]$. See primer and Ch. 4.

51 Notation Summary

This is a one-page summary of the notation used throughout the book.

51.1 Strings and characters

Symbol	Meaning
S, T	strings
$n = \|S\|$	length of S
$S[i]$	i-th character, 0-indexed
$S[l..r]$	substring from l to r inclusive
$S[l..]$	suffix starting at l
Σ	alphabet
$\sigma = \|\Sigma\|$	alphabet size
$\mathrm{rev}(S)$	reversal of S
$\mathrm{rot}_k(S)$	k-th cyclic rotation
ε	empty string

51.2 Structures

Symbol	Meaning
$\pi[i]$	prefix function value at i
$z[i]$	Z-function value at i
$\mathrm{sa}[i]$	suffix array index
$\mathrm{lcp}[i]$	LCP of adjacent SA suffixes
$\mathrm{rank}[i]$	inverse of sa
$\mathrm{link}(v)$	suffix link of SAM / eertree state v
$\mathrm{len}(v)$	max length of strings in class v
$\mathrm{endpos}(v)$	end position set of state v
$\mathrm{fail}(v)$	AC failure link
$\delta(v, c)$	automaton transition
$\mathrm{BWT}[i]$	BWT character at position i
$\mathrm{Occ}(c, i)$	number of c in $\mathrm{BWT}[0..i-1]$
$\mathrm{LF}(i)$	LF-mapping: BWT $\to$ first column
w	machine word size (typically 64)

51.3 Complexity notation

Symbol	Meaning
$\mathcal{O}(f(n))$	asymptotic upper bound
$\Theta(f(n))$	tight bound
$\Omega(f(n))$	lower bound
$\tilde{\mathcal{O}}(f(n))$	$\mathcal{O}(f(n)\log^k n)$ for small k
H_0	0-order empirical entropy of S
$[P]$	Iverson bracket: 1 if P true, 0 else

52 Master Cheat Sheet and Decision Tree

This is the single page to glance at during a contest. It combines the primer's table with all the new expert algorithms.

52.1 Decision table

Problem signature	Algorithm	Complexity	See Ch.
1 pattern, 1 text	KMP / Z / Shift-Or	$\mathcal{O}(n+m)$	§3, §4, §11
Many patterns, 1 text	Aho–Corasick	$\mathcal{O}(\sum \lvert P\rvert + \lvert T\rvert)$	§5
Count strings avoiding patterns	AC + matrix exponentiation	$\mathcal{O}(\lvert Q\rvert^3 \log n)$	§5
Many substring-equality queries	Hashing (random base)	$\mathcal{O}(n)$ + $\mathcal{O}(1)$/q	§2
LCS of 2 strings	SAM walk / SA + LCP	$\mathcal{O}(\lvert S_1\rvert+\lvert S_2\rvert)$	§7, §6
LCS of k strings	Generalised SAM / SA + LCP	$\mathcal{O}(\sum \lvert S_i\rvert)$	§9, §6
Distinct substrings (offline)	SA + LCP	$\mathcal{O}(n \log n)$	§6
Distinct substrings (online)	SAM	$\mathcal{O}(n)$	§7
Smallest cyclic rotation	Booth / Duval / SAM walk	$\mathcal{O}(n)$	§15
Smallest lexicographic suffix	Duval last factor	$\mathcal{O}(n)$	§15
Longest palindromic substring	Manacher	$\mathcal{O}(n)$	primer
Minimum palindromic factorisation	Eertree + series links	$\mathcal{O}(n \log n)$	§13
All palindromic substrings online	Eertree	$\mathcal{O}(n)$	primer
Substring queries, online	SAM	$\mathcal{O}(n)$ + $\mathcal{O}(\lvert T\rvert)$/q	§7
Substring comparison, many queries	SA + LCP + sparse table	$\mathcal{O}(n \log n)$ + $\mathcal{O}(1)$/q	§6
Compressed full-text index	BWT + FM-index + wavelet	$\mathcal{O}(n)$ + $\mathcal{O}(\lvert P\rvert \log \sigma)$/q	§10
Max XOR pair / max XOR with x in range	Persistent trie	$\mathcal{O}(n \log V)$ + $\mathcal{O}(\log V)$/q	§14
k-th smallest in range	Persistent trie	$\mathcal{O}(n \log V)$ + $\mathcal{O}(\log V)$/q	§14
Edit distance, $\lvert B\rvert \le 64$	Myers bit-vector	$\mathcal{O}(\lvert A\rvert)$	§11
k-mismatch, $\lvert P\rvert \le 64$	Bit-parallel k-mismatch	$\mathcal{O}(\lvert T\rvert)$	§12
k-differences (edit), $\lvert P\rvert > 64$	Landau–Vishkin	$\mathcal{O}(\lvert T\rvert \cdot k)$	§12
All squares in S	Runs enumeration	$\mathcal{O}(n)$	§15
Count distinct substrings over tree paths	Trie + small-to-large merge	$\mathcal{O}(n \log^2 n)$	§14

52.2 Decision flowchart (text form)

Run through the following questions in order; stop at the first that fires.

Q1. Is the problem about a single pattern matching in a single text? **Yes** → KMP, Z, or Shift-Or if $\lvert P\rvert \le 64$ (Ch. 3, 4, 11).

Q2. Is the problem about many patterns matching in one text? **Yes** → Aho–Corasick (Ch. 5). If counting strings of length n avoiding patterns: AC + matrix exponentiation.

Q3. Is the problem about *all* substrings of S (distinct count, k-th, longest repeated)? **Yes** → SA + LCP + sparse table (Ch. 6). If S is online, SAM.

Q4. Is the problem about substring equality (many queries)? **Yes** → Hashing with random base (Ch. 2).

Q5. Is the problem about cyclic rotations? **Yes** → Booth or Duval (Ch. 15).

Q6. Is the problem about palindromes? **Yes** → Manacher (longest, count); Eertree + series links (factorisation) (Ch. 13).

Q7. Is the problem about multiple strings and their common substrings? **Yes** → Generalised SAM (Ch. 9); or SA + LCP on the concatenated string.

Q8. Is the problem about approximate matching (k errors)? **Yes** → Myers bit-vector (small $|P|$) or Landau–Vishkin (large $|P|$, small k) (Ch. 12).

Q9. Is the problem about compressed full-text search or bioinformatics alignment? **Yes** → BWT + FM-index (Ch. 10).

Q10. Is the problem about XOR / k-th in range / max XOR with x? **Yes** → Persistent trie (Ch. 14).

Q11. Is the problem about squares, runs, or all maximal repetitions? **Yes** → Runs enumeration (Ch. 15).

Q12. Otherwise: re-read the problem. Many CP string problems are reducible to one of the above via a clever reformulation.

52.3 Hybrid problems cheat sheet

Hybrid pattern

A condensed list of the canonical hybrid patterns, with the Codeforces problem exemplar for each:

- **AC + DP** → CF 1202E (count pairs A, B such that $A + B \subseteq T$).
- **AC + matrix exponentiation** → CF 86C (count strings of length n containing a pattern).
- **SA + LCP + sparse table** → CF 1066F, CF 1090J.
- **SAM + suffix-link tree DP** → CF 235C, CF 204E.
- **GSAM + bitmask propagation** → CF 204E.
- **Hashing + binary search on LCP** → CF 514C, CF 1266C.
- **Eertree + series-link DP** → CF 932G.
- **BWT + FM-index + wavelet** → bioinformatics problems, less common in CP.
- **Bit-parallel + Myers** → CF 5E, edit distance problems.
- **Trie + small-to-large merge** → CF 778C, problems on string-labelled trees.

52.4 Reading order for the expert

If you are coming to the expert edition fresh from the primer, read in this order: Chapter 2 (hashing with random base, ∼ 4 pages), Chapter 3 (failure automaton and periodicity, ∼ 5 pages), Chapter 6 (SA + sparse table, ∼ 5 pages), Chapter 7 (suffix-link tree DP, ∼ 4 pages), Chapter 8 (suffix tree, ∼ 4 pages), then the new material: BWT (Ch. 10), bit-parallel (Ch. 11), approximate (Ch. 12), eertree factorisation (Ch. 13), runs (Ch. 15).

Annotated Bibliography

This annotated bibliography collects the canonical references for every algorithm in this book. Each entry includes a brief description and a recommendation on when to consult it.

Textbooks and reference works

- **M. Crochemore, W. Rytter, *Jewels of Stringology*** (Springer, 2003). The definitive textbook on advanced string algorithms. Covers periodicity, runs, Lyndon words, palindromic factorisation, and the fine structure of suffix trees and suffix arrays. Recommended for Chapters 32, 34, 13.
- **D. Gusfield, *Algorithms on Strings, Trees, and Sequences*** (Cambridge, 1997). The classical bioinformatics textbook; focuses on suffix trees and their applications to genomics. Recommended for Chapters 8, 26, 27.
- **Antti Laaksonen, *Competitive Programmer's Handbook***, Chapter 14 (free PDF). The gentle introduction to CP string algorithms; the natural pre-reading for this book.
- **G. Navarro, *Compact Data Structures: A Practical Approach*** (Cambridge, 2016). Modern reference for compressed indexes, wavelet trees, and the FM-index. Recommended for Chapter 21.
- **cp-algorithms.com — String algorithms section** (link). The most useful CP reference; every algorithm in this book has a worked example there.

Foundational papers

- **D. E. Knuth, J. H. Morris, V. R. Pratt**, "Fast pattern matching in strings", *SIAM J. Comput.* 6(2):323–350, 1977. The original KMP paper. Read for the failure-link amortised analysis.
- **A. V. Aho, M. J. Corasick**, "Efficient string matching: an aid to bibliographic search", *Comm. ACM* 18(6):333–340, 1975. The original Aho–Corasick paper.
- **E. Ukkonen**, "On-line construction of suffix trees", *Algorithmica* 14(3):249–260, 1995. The canonical suffix-tree algorithm.
- **Udi Manber, G. Myers**, "Suffix arrays: a new method for on-line string searches", *SIAM J. Comput.* 22(5):935–948, 1993. The original suffix array paper.
- **J. Kärkkäinen, P. Sanders**, "Simple linear work suffix array construction", *ACM TALG* 9(1:6), 2003. The DC3 algorithm.
- **G. Nong, S. Zhang, W. H. Chan**, "Linear suffix array construction by almost pure induced-sorting", *SPIRE* 2009. The SA-IS algorithm.
- **M. Crochemore, L. Ilie, W. Smyth**, "A simple algorithm for computing the LCP array in $\mathcal{O}(n)$ time", 2008. The cleanest modern write-up of Kasai's algorithm.
- **A. Blumer et al.**, "The smallest automaton recognizing the subwords of a given word", *Inf. Process. Lett.* 21(4):181–185, 1985. The original suffix automaton paper.

Bit-parallel and approximate matching

- **R. Baeza-Yates, G. H. Gonnet**, "A new approach to text searching", *Comm. ACM* 35(10):74–82, 1992. The Shift-Or algorithm.

- **G. Navarro, M. Raffinot**, "A bit-parallel suffix automaton for pattern matching", *SPIRE* 1998. BNDM.
- **E. W. Myers**, "A fast bit-vector algorithm for approximate string matching based on dynamic programming", *J. ACM* 46(3):395–415, 1999. The Myers bit-vector edit distance.
- **G. M. Landau, U. Vishkin**, "Fast parallel and serial approximate string matching", *J. Algorithms* 10(2):157–169, 1989. The Landau–Vishkin k-error algorithm.
- **A. Backurs, P. Indyk**, "Edit distance cannot be computed in strongly subquadratic time (unless SETH is false)", *STOC* 2015. The conditional lower bound.

Compressed indexes

- **M. Burrows, D. J. Wheeler**, "A block-sorting lossless data compression algorithm", DEC SRC Research Report 124, 1994. The original BWT.
- **P. Ferragina, G. Manzini**, "Indexing compressed text", *J. ACM* 52(4):552–581, 2005. The FM-index.
- **R. Raman, V. Raman, S. S. Rao**, "Succinct indexable dictionaries with applications", *WADS* 2002. RRR rank/select structure.
- **G. Navarro, V. Mäkinen**, "Compressed full-text indexes", *ACM Computing Surveys* 39(1), 2007. The canonical survey.

Palindromes, periodicity, and runs

- **M. Rubinchik**, "Eertree — palindromic tree", codeforces.com/blog/entry/13959. Original eertree write-up.
- **A. Eppstein et al.**, "Tracking the minimum palindromic factorisation", 1992. Palindromic factorisation via series links.
- **R. Kolpakov, G. Kucherov**, "Finding maximal repetitions in a word in linear time", *FOCS* 1999. The runs theorem.
- **N. J. Fine, H. S. Wilf**, "Uniqueness theorems for periodic functions", *Proc. AMS* 16(4):821–825, 1965. The Fine–Wilf periodicity lemma.
- **J. P. Duval**, "Factorizing words over an ordered alphabet", *J. Algorithms* 4(4):363–381, 1983. The Duval Lyndon factorisation algorithm.
- **K. S. Booth**, "Lexicographically least circular substrings", *Inf. Process. Lett.* 10(4–5):240–242, 1980. Booth's algorithm.

Online resources

- **cp-algorithms.com — String algorithms**. The single most useful CP reference for string algorithms.
- **Codeforces blog entry 11333 — Emax on string algorithms**. Long-form exposition by user `Emax`, with worked Codeforces examples.
- **Codeforces blog entry 16784 — indy256 on SAM**. Suffix automaton tutorial with applications.
- **Codeforces blog entry 18169 — Burunduk1 on suffix structures**. Suffix structures comparison.

- **Wikipedia — String-searching algorithm**. Good for quick definition lookups.
- **stringology.org**. The Stringology research community's page; proceedings of Prague Stringology Conference (PSC) and predecessors.

Research-paper venues for follow-up reading

- **CPM** — Combinatorial Pattern Matching. The main string-algorithms conference.
- **SPIRE** — String Processing and Information Retrieval.
- **SODA** — ACM-SIAM Symposium on Discrete Algorithms (broader; many string papers).
- **PSC** — Prague Stringology Conference (formerly Prague Stringology Club Workshop). The focused stringology venue.
- **ICALP** — International Colloquium on Automata, Languages and Programming. Track B for stringology.

End of the expert edition. You now have the full toolbox: from the workhorses (KMP, hashing, SA, SAM) through the specialised structures (Ukkonen, GSAM, BWT/FM-index, eertree factorisation) to the bit-parallel era (Myers, Landau–Vishkin) and the runs theorem. The next level is original research — papers in SODA, SPIRE, and CPM.